

SkyGuard Blueprint

Real-time anomaly detection for Automatic Weather Stations from temperature, pressure and humidity alone. A build plan for a system that does not just flag a bad value — it names which sensor failed, why, how sure it is, and when it will need servicing.

Read this first.

This is the *plan*, and it is kept because the reasoning in it is still the reasoning — the drift argument, the physics, where the accuracy comes from, the traps. For what is actually running, read **SkyGuard As-Built**, which was written by reading the code and querying the deployed service. Where the two disagree, the as-built is right.

The problem statement

ID 26073 ORG MINISTRY OF EARTH SCIENCES (MOES) DEPT INDIA METEOROLOGICAL DEPARTMENT CATEGORY SOFTWARE THEME DISASTER MANAGEMENT
Problem Statement Title AI/ML-Based Intelligent Anomaly Detection for Automatic Weather Stations (AWS) Title SkyGuard AI: Intelligent Real-Time Anomaly Detection System for Temperature, Pressure, and Humidity Sensors in Automatic Weather Stations Background Automatic Weather Stations (AWS) are critical components of modern meteorological observation networks. These stations continuously monitor atmospheric parameters and provide real-time data for weather forecasting, climate monitoring, disaster management, aviation, agriculture, and scientific research. However, AWS observations often contain anomalies caused by sensor malfunction, communication failures, calibration drift, power fluctuations, harsh environmental conditions, and data corruption. Erroneous observations can significantly impact weather forecasting accuracy and decision-making systems. Traditional threshold-based quality control methods are often insufficient for identifying complex or hidden anomalies in meteorological data streams. Problem Statement Develop an AI/ML-based intelligent anomaly detection system capable of automatically identifying abnormal, inconsistent, or faulty observations from Automatic Weather Stations in real time using only the following parameters:

- Temperature (°C)
- Atmospheric Pressure (hPa)
- Relative Humidity (%)

The system should distinguish between genuine meteorological events and sensor/data anomalies while minimizing false alarms and enabling scalable deployment across large weather observation networks.

Objectives

- Detect anomalies in real-time AWS data streams.
- Identify sensor faults, spikes, frozen values, and communication errors.
- Learn normal temporal and seasonal patterns of temperature, pressure, and humidity.
- Perform multivariate consistency analysis among atmospheric parameters.
- Provide confidence scores and explainable AI-based reasoning for detected anomalies.
- Predict possible sensor degradation and maintenance requirements.
- Optionally suggest corrected/imputed values for anomalous observations.

Expected Inputs

Participants may use historical AWS datasets, simulated anomalies, or streaming sensor data containing the following meteorological parameters:

- Temperature – °C
- Atmospheric Pressure – hPa
- Relative Humidity – %

Expected Outputs

- Real-time anomaly alerts
- Severity and confidence scores
- Root-cause classification
- Visualization dashboard
- Sensor health status
- Corrected data estimation (optional)

Suggested Technologies

- Explainable AI (SHAP/LIME) (Preferable)
- Edge AI for low-power deployment on ESP32

Evaluation Criteria *(To be evaluated in anomaly injected data)*

- Innovation & Novelty – 25%
- Detection Accuracy – 20%
- Real-Time Capability – 15%
- Explainability – 10%
- Scalability – 10%
- Practical Deployability – 10%
- Visualization/UI – 5%

- Energy Efficiency – 5%

Example Use Case

An AWS suddenly reports a temperature of 55°C with extremely high humidity and abnormal pressure variation while neighboring stations show normal conditions. The AI system should analyze temporal and spatial consistency, identify the reading as a probable sensor anomaly, generate an alert, and suggest corrective action.

Grand Challenge

Can AI build a self-aware and self-healing weather observation network capable of delivering trustworthy atmospheric data under all environmental conditions?

Output

Fully executable code with example usage and a document explaining various use cases.

MVP — what actually makes this different

Everything in this section is a *reframe a judge has not heard*, not a number. The measured work is at the end of this document; this is the pitch surface.

The position, in one line.

Everyone else builds a better buddy check. We measure ours against IMD's **own** 2015 spatial regression test — then solve the station it structurally cannot reach, using a reference clock the station already carries in its own barometer.

The ML spine.

Everyone puts the ML in the anomaly scorer. **We put it in the reference.** The model's job is to predict what this station *should* have read; the anomaly is the subtraction. This is the defensible framing because a better reference improves every fault class at once while a fancier scorer improves none — and it follows from the arithmetic, since drift latency is governed by the noise of the reference, not by the cleverness of the detector.

The differentiators

#	IDEA	THE LINE TO SAY	WHY NOBODY ELSE HAS IT
1	Trust map	"Every other map tells you which station is broken. Ours tells you which stations you have no way of knowing about."	Requires admitting a structural limit. The output is a prioritised list of where to send a reference instrument.

#	IDEA	THE LINE TO SAY	WHY NOBODY ELSE HAS IT
2	Fleet, not station	<i>"We don't diagnose stations. We diagnose fleets."</i>	IMD buys by tender lot, so failures cluster by vendor, firmware and install year — and a whole lot drifting together is invisible to every neighbour method, IMD's included.
3	The barometer is a clock	<i>"We didn't add a clock to the station. We found the one already there."</i>	The semidiurnal pressure tide gives a per-station phase reference. Clock drift is otherwise undetectable: the series stays internally perfect while every downstream time-matched comparison rots.
4	One fault, one work order	<i>"A broken thermometer makes the hygrometer look broken too. We tell you which one to actually replace."</i>	T, P and RH are one thermodynamic state, not three signals. A naive pipeline raises two tickets for one broken sensor.
5	Detection that reaches backwards	<i>"We tell you which records in your archive are now suspect, and since when."</i>	Onset dating with an interval, emitted as a re-flag manifest. This matters most to the custodian of a climate record.
6	The do-not-dispatch call	<i>"The most valuable thing this system does is tell you when NOT to send the van."</i>	Splits every work order into truck-roll versus a central coefficient fix.
7	The station's digital twin	<i>"We don't ask a model whether a reading looks weird. We ask what it should have been."</i>	A conditional generative model of the station given its neighbours, hour and season. Its own uncertainty sets the threshold.
8	Learned station graph	<i>"We let the data decide who counts as a neighbour."</i>	A graph network learns edge weights, so a station 40 km away across the Ghats is correctly down-weighted against one 200 km away on the same plain.
9	Physics-informed	<i>"Our model cannot predict something the atmosphere cannot do."</i>	Magnus, the dew-point bound and humidity conservation enter as hard constraints, not as hopeful learning.
10	Station fingerprints	<i>"Every station has a signature. We watch it change."</i>	A learned embedding per station; degradation shows as migration through embedding space, and the direction indicates which fault.
11	Generate the missing corpus	<i>"There is no labelled Indian AWS fault dataset. So we made one, and we're releasing it."</i>	The only deliverable here with a life after the hackathon.
12	It asks for its own labels	<i>"Day one it has no labels. Month six it has thousands, and it asked for</i>	Every operator Confirm/Reject is a label; active learning chooses which alerts to adjudicate first.

#	IDEA	THE LINE TO SAY	WHY NOBODY ELSE HAS IT
		the ones that mattered.”	

Recommended spine for a six-slide deck: 1, 2 and 3 make a judge reconsider the *problem* rather than admire the solution, and they reinforce each other — all three concern the limits of neighbour-based QC, which is exactly what IMD already runs. Then 4 and 5 as the operator payoff, with the ML spine carrying 7–10 underneath.

What we refuse to build, and the line for each

REFUSED	THE JUSTIFICATION
LLM narration of alerts	“Our confidence comes from the detector's calibration, not a language model's phrasing. We did not want a hallucination between the sensor and the meteorologist.” If an LLM is used at all it renders already-computed fields into a sentence, and never decides, scores or explains.
Auto-correcting the archive	“The observation record is the archive. We flag and suggest; we never silently rewrite what the instrument reported.”
A new flag taxonomy	Adoption cost. IMD grades flags 0–5; anything else is a migration project rather than a tool.
Remaining-useful-life prediction	Requires years of labelled failure history nobody has. We report drift magnitude and rate; the instrument engineer decides.
A hardware AWS as the product	IMD's stations are already in the ground and WMO-calibrated. The product must work on the data they already send.
Kafka / Spark / a cluster	A national network at 15-minute cadence is a couple of records per second. Proposing a cluster proves the arithmetic was never done.

The limits we state before being asked

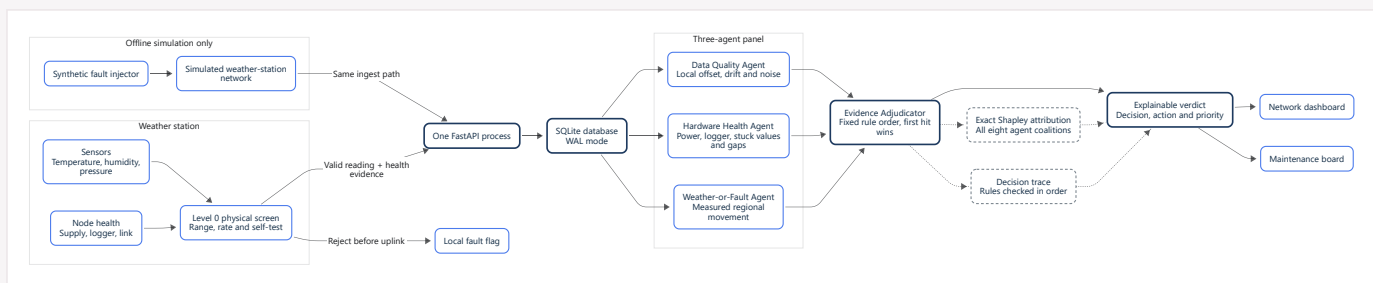
- A neighbourless station reading a **uniform** offset with correct diurnal shape is undetectable from its own data. No method fixes this; the honest response is the trust map of item 1.
- A drift affecting the **entire** network is unidentifiable without an absolute reference, by construction.
- The pressure tide buys the **clock and nothing else** — a pressure span error moves it far too little to see, and an additive offset leaves it untouched entirely.
- **IMD operates 1,000+ AWS**, plus ARGs and manual observatories. Do not inflate this figure; a MoES judge knows it.

Status. Items 1–6 are design positions, some resting on parts already built; 7–12 are proposed and unbuilt. Nothing in this section is measured — the measured work is in “Measured so far” at the end.

The architecture, drawn

Three views of the same system: what it is, how it reasons, and what an operator does about it. They are rendered on a white ground deliberately — these are the figures intended for the submission deck, and they should look the same on a projector as on this page.

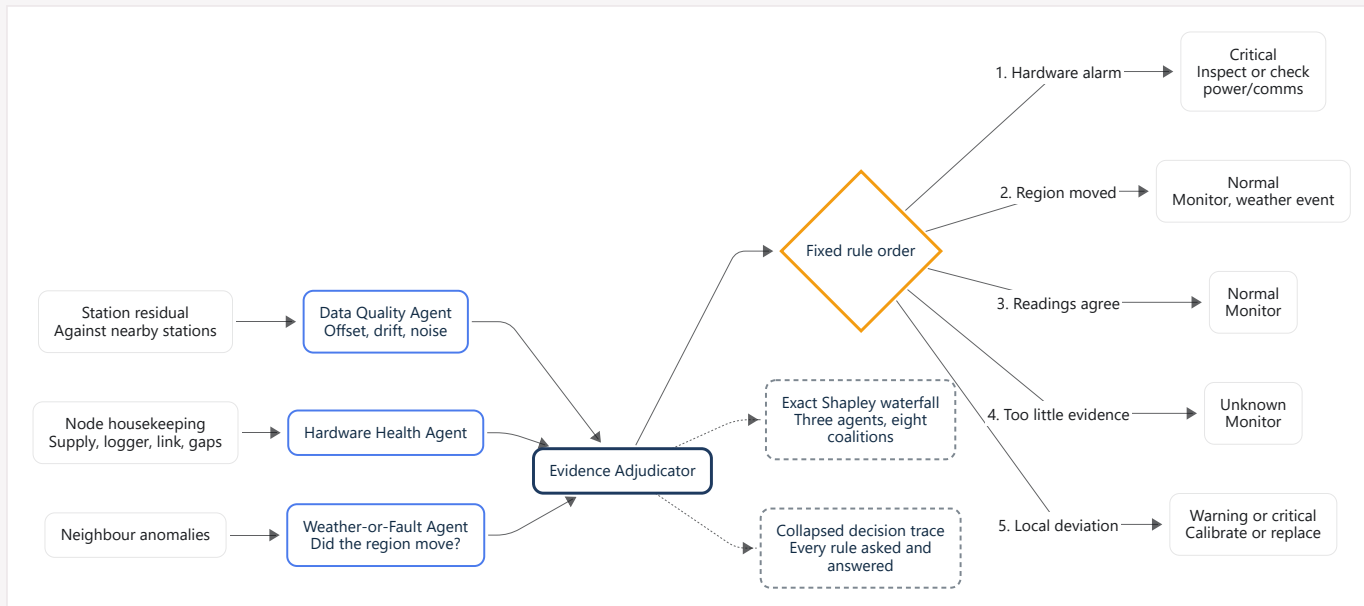
The system end to end



Station tier through to operator output, and the two things worth noticing. The **level-0 screen sits on the station**, so a physically impossible reading is rejected before it costs any bandwidth — that check used to be a central one. And the **panel is the shared door**: the simulated network and the live node are judged by the same three agents and the same adjudicator, which is what stops two screens disagreeing about one sensor.

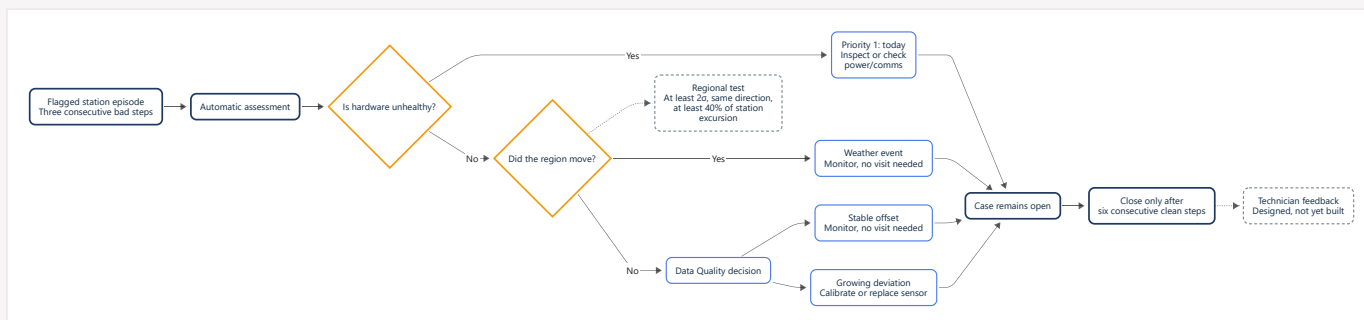
One edge here is the intent, not the code. The simulated network is drawn arriving through ingest and the store like everything else. Today it does not: it is a pre-built export the panel reads directly, and `store.record()` is called only from `api/ingest.py`. Routing the simulation through the same door is the correct shape, is agreed, and is deliberately deferred rather than forgotten.

The reasoning layer



Three agents over non-overlapping evidence feed one adjudicator, which checks a fixed order of rules and stops at the first that fires. The dashed paths are the explanation: every verdict is re-run on all eight coalitions of the panel to give each agent an exact Shapley value and a counterfactual, and the adjudicator records the walk it took. **There is no learned model anywhere in this path.**

What the operator does



The loop closes twice. A confirmed fault becomes a work order with a recommended action; a rejected one is **marked as a weather event and monitoring continues**, rather than being silently dropped. Either way the outcome feeds back to the model, which is how a system with no labels on day one acquires them.

The AI architecture — three agents and an adjudicator

The problem statement asks whether AI can build a “self-aware and self-healing weather observation network”. That phrase has to cash out as an architecture or it is decoration.

This is the architecture, and the word *agent* here means an autonomous specialist checker with its own evidence source – **not** a language model.

Drawn in *The architecture*, drawn above, under “The reasoning layer”. The five-checker design this replaced – physics, neighbour, history, instrument, clock – was never implemented that way: the physics screen moved onto the node, and the remaining evidence collapsed into three agents that genuinely look at different things. Fewer players is also what makes the attribution exact rather than sampled: eight coalitions is enumerable, thirty-two is not worth it.

The belief state – what the system actually remembers

A pipeline judges each reading from nothing and forgets it. The architectural change is that **every sensor carries a file that persists and updates**. Roughly 1,000 stations × 3 sensors is ~3,000 of these, each a few dozen bytes.

WHAT IT HOLDS	WHAT IT MEANS	WHAT IT ANSWERS IN THE PROBLEM STATEMENT
bias	how far off this sensor currently reads	“corrected/imputed values”
drift rate	how fast that offset is growing	“predict possible sensor degradation”
noise scale	how jittery it has become	degradation shows in the second moment before the first
trust	how much the network currently believes it	“sensor health status”
last service	when it was touched, and by whom	stops a repaired station staying condemned by its own history

The anomaly becomes a by-product of the update, not the goal of the computation. That single inversion is what makes the separate objectives in the statement one mechanism instead of seven features: degradation prediction is reading the drift rate off the file, the confidence score is the state's own uncertainty, and the root cause is whichever part of the file moved.

The nearest operational precedent is **variational bias correction**, which ECMWF has run since 2006: each instrument's bias coefficients sit inside the analysis control vector and are estimated jointly with the atmospheric state, every cycle, rather than computed offline. The architecture is twenty years proven – what is unclaimed is turning it toward *diagnosing the instrument* rather than *correcting the forecast*.

Which tier may reason about what

The split is forced by information, not convenience. A station has no neighbours; it has a radio. The rule that makes it principled: **a tier may only run inference over information it actually possesses.**

TIER	INFORMATION AVAILABLE	THEREFORE IT CAN DECIDE
Station node ESP32-class	its own three sensors, its own recent history, right now	Range, rate, stuck values, supply health. Not whether its temperature is right in an absolute sense — it has no way to know, so no model there should pretend otherwise
Server, streaming	every station, this cycle	Everything needing a neighbour: the checkers, the adjudicator, trust, alerting
Offline, batch	years of history plus external fields	Baseline and tide re-fits, the neighbour graph, lot-level comparisons, threshold re-calibration

The healing loops, and the brakes on each

“Self-healing” means the system takes actions on itself with no human in the loop. That is only safe if every loop has a governor, because **a system that heals wrongly is worse than one that does nothing.** Each loop below is a controller with a trigger, an action, and something that stops it running away.

LOOP	ACTION TAKEN	HOW IT COULD GO WRONG	THE BRAKE
Reference exclusion	A distrusted station stops being used as a yardstick for its neighbours	A real regional event makes everyone disagree, so everyone excludes everyone — during exactly the weather the data matters most	Subtract the neighbourhood's common-mode movement first; only the residual counts against trust. Never drop below a minimum number of references — below it, the station is <i>unjudgeable</i> , not condemned
Trust update	Belief in a station moves with its recent agreement	Trust collapses to an absorbing state and a station that had one bad week is never used again	A floor, so nothing is excommunicated, and asymmetric recovery — fast to distrust, slow to re-trust, but never permanent
Correction and imputation	Publishes an estimate alongside a suspect reading	The killer. Corrects away a genuine heat wave or pressure crash, and the event is lost	Corrections are additive and reported beside the raw value, which is never overwritten ; magnitude is rate-limited; beyond the cap the system escalates instead of correcting. <i>When unsure, degrade to flagging, not to fixing</i>

LOOP	ACTION TAKEN	HOW IT COULD GO WRONG	THE BRAKE
Post-service reset	A serviced station's baseline is re-fitted and its history set aside	A fault that mimics a repair launders itself back into trust	Only a confirmed work order triggers a full reset; a merely detected one widens uncertainty and asks for confirmation
Budget control	The alert threshold tracks a set volume the operations desk can action	Suppresses alarms during a genuine severe-weather outbreak, to stay within budget	The controller is disabled whenever the network-wide common-mode signal is large. Severe weather puts the system into report-everything mode

Four of those five brakes are the same instruction: fall back to honest uncertainty. That is the thesis in operational form, and it is what makes an autonomous system safe enough to put on a national network.

What each checker interrogates

AGENT	ITS EVIDENCE	THE QUESTION IT ANSWERS
Physics	thermodynamics	Can this state exist at all?
Neighbour	surrounding stations	Did the region move, or just this one?
History	the station's own past	Is this what this station always does?
Instrument	fault signatures	Does this match a known failure mode?
Clock	the pressure tide	Is this station's time even right?

The line to say out loud.

“We don't have one model deciding. We have five specialists that must agree – and a system honest enough to say ‘unknown’ when they don't.”

What is borrowed, what is ours, and how it gets tested

This design is a **rediscovery, three times over**, and saying so is stronger than claiming invention – it means the pattern can be cited rather than defended from scratch.

WHAT WE ARE DOING	WHAT IT IS ALREADY CALLED	SOURCE
Independent checkers, a shared record, a control component that decides	Blackboard architecture	Hearsay-II (Erman, Hayes-Roth, Lesser & Reddy, 1980); Nii, <i>AI Magazine</i> 1986. Presented as a paradigm for “ <i>errorful knowledge-intensive problems requiring multiple, cooperating knowledge sources</i> ”
Combining several residual checks on weather observations into one decision	Complex Quality Control (CQC)	Gandin, <i>Monthly Weather Review</i> 116(5), 1988. Operational at the Moscow Hydrometeorological Centre and in NCEP practice
Each check sensitive to some faults and blind to others, read as a pattern	Structured residuals / fault signature matrix	Gertler, analytical redundancy in fault detection and isolation
Returning “unknown” instead of a verdict	Selective prediction / reject option; the <i>suspect flag</i>	WMO <i>Guidelines on QC Procedures for Data from AWS</i> ; Geifman & El-Yaniv, ICML 2019

Gandin 1988 is the one that matters. Combining a complex of residuals rather than thresholding one is the founding idea of meteorological QC, and it is nearly forty years old. **Presenting the five-checker layout as novel to a meteorologically literate panel would be a credibility problem, not an innovation score.** The correct framing is: we apply the established CQC pattern, and here is specifically what we add to it.

The rule as first written was not falsifiable. This is the fix.

The original wording – “*corroboration must come from agents whose evidence does not overlap*” – asserts independence rather than measuring it, and the ensemble literature is emphatic that assumed diversity does not deliver.

- **Kuncheva & Whitaker (2003)** tested ten diversity measures and found the relationship between diversity and ensemble accuracy “*weak and inconsistent on real problems*”. Accuracy does not rise monotonically with diversity under any of them.
- **Knight & Leveson (1986)** had 27 programs written independently from one specification and ran a million test cases. Versions failed *together* far more often than independence predicts, correlated through shared interpretation of an ambiguous spec. The result has survived forty years of attack.
- **Common-cause failure has a floor.** In reliability engineering the beta-factor model puts deliberately diverse redundant subsystems at roughly 10^{-3} . Diversity buys about one order of magnitude and cannot be driven to zero.

The rule, restated so it can be checked: corroboration must come from checkers whose **measured double-fault rate under the prevailing weather regime** is below a stated threshold – not from checkers we believe to be independent.

The deliverable is a 6×6 error-correlation and double-fault matrix computed on labelled data and **published**, reported separately for calm and for extreme conditions. Independence that holds on average and collapses in the tail is exactly the failure that matters, and a single aggregate number hides it.

Our own concrete common causes, written down rather than discovered later: physics, spatial and temporal checks may all reduce to a departure from the same background field; severe weather breaks the spatial *and* temporal smoothness assumptions at the same moment; one wrong station elevation poisons the physics, spatial and pressure-tide checks together; and one person writing all six checkers to one internal specification correlates their blind spots by construction.

How it must be evaluated

There is **no benchmark discipline for multi-agent systems** – current surveys say evaluations are run in narrow bespoke scenarios and are not comparable. So a reviewer cannot ask for a standard MAS benchmark. They *can* ask why we ignored the three established methodologies that do apply.

QUESTION	ESTABLISHED METHOD
Does the panel beat its best single member?	Ablation table: every checker alone, best single, simple OR, majority vote, and the oracle upper bound. Then leave-one-checker-out – a checker whose removal costs nothing is not earning its place
Is the difference real?	Wilcoxon signed-rank for two systems, Friedman with post-hoc and critical-difference diagrams across many, treating stations and seasons as the datasets (Demšar, <i>JMLR</i> 2006)
Is the abstention honest?	Risk-coverage curve, with coverage reported in every result and baselines compared at matched coverage
Are the checkers actually independent?	Double-fault and Q-statistic matrix, stratified by regime (Kuncheva & Whitaker, 2003)

An untethered UNKNOWN makes any system look perfect. A panel that abstains on 95 % of readings will show a beautiful selective risk. Coverage travels with every number or the number means nothing — this is the most likely line of attack on the whole design.

And the outcome to be ready for: **if the panel does not beat its best single checker on that ablation table, the correct action is to ship the best single checker.** The ensemble literature says that is a live possibility, not a remote one.

On combining the verdicts

Dempster-Shafer theory is the obvious formalism, because mass on the whole frame represents ignorance explicitly — which is what UNKNOWN needs. **But Dempster's rule breaks precisely on our conflict path.** Zadeh's counterexample: two sources that almost entirely reject a hypothesis, disagreeing sharply on everything else, combine to give that rejected hypothesis belief 1.0 — because the normalisation divides out the conflict and amplifies whatever rounding error survived. High conflict is exactly when we want *more* ignorance, and the rule delivers false certainty instead.

Use **Yager's rule**, which reassigns conflict mass to ignorance, or reliability-weighted averaging before combination — or keep a transparent signature-matrix rule an operational meteorologist can read, and borrow only the belief/plausibility vocabulary.

So what is actually ours

Stripping out everything that is CQC, blackboard or FDI under another name, what remains is specific and defensible: **the semidiurnal pressure tide used as a per-station reference clock**, which detects timing faults with no neighbour involved and is measured viable on our data; the **Level-0 screen on the station itself**, which IMD's own 2015 paper records as not implemented; and the **work order with a dated onset** rather than a flag. Those are worth more than an architecture claim, and they were being buried under one.

Grounding: this is a recognised class of system

Condition monitoring of a deployed fleet is an established engineering discipline with its own international standard, and the problem statement's objectives map onto it closely enough that whoever wrote them very likely knows it. Naming the standard is worth more than inventing a private architecture.

ISO 13374 PROCESSING BLOCK	WHAT PS26073 ASKS FOR
Data Acquisition	“real-time AWS data streams”
Data Manipulation	“learn normal temporal and seasonal patterns”
State Detection	“detect anomalies ... sensor faults, spikes, frozen values, communication errors”
Health Assessment	“sensor health status”
Prognostic Assessment	“predict possible sensor degradation and maintenance requirements”
Advisory Generation	“suggest corrective action”, “corrected/imputed values”, “visualization dashboard”

Cite this carefully. Verified from the standard's own cover and contents: **ISO 13374-1:2003**, *Condition monitoring and diagnostics of machines – Data processing, communication and presentation – Part 1: General guidelines*, first edition 2003-03-15, whose clause 2.2 is “Data-processing blocks” and whose Annex A is about MIMOSA. MIMOSA, the custodian of OSA-CBM, states there are **six** functional blocks – the “seven layers” commonly drawn in papers is a rendering, not the standard. The six block names above are taken from MIMOSA and the CBM literature; the free ISO preview is front matter only, so they have **not** been read from the normative clause. Say “per MIMOSA and the condition-monitoring literature”, never “per ISO clause 2.2”, unless someone buys the standard.

Two clauses in that contents page are worth more than the block list and nobody quotes them: **4.2 “Determination of work flow procedures”** and **5 “Responsible personnel”**, which names Operators, Operations engineer, Reliability analyst and Management. The standard makes *workflow and named human roles* part of what a condition-monitoring system is. That is institutional backing for delivering a work order rather than a flag – it is not our preference, it is what the standard says the system is for.

The refusal, stated on the slide rather than discovered in questioning. No language model sits anywhere in the decision path. Confidence here comes from the detectors' own calibration, not from a model's phrasing, and an LLM between the sensor and the meteorologist is a hallucination surface on live operational data. If one appears at all it renders an already-computed verdict into a sentence – it never decides, scores, or explains.

What four rounds of research changed

This blueprint was reviewed against the meteorological QC literature and pressure-tested on method selection, explainability and operator workflow. Three findings changed the design materially, and one of them was a factual error. They are stated here rather than quietly fixed, because the corrections are more instructive than the original plan.

Correction 1 — the dew-point check was a tautology. The earlier draft listed $T_d \leq T$, $RH \leq 100\%$ and $e \leq e_s(T)$ as three independent physics constraints. When T_d is computed from T and RH via Magnus, they are **algebraically the same inequality**. Substituting $RH = 100$ into Magnus returns $T_d = T$ exactly, at every temperature; $RH < 100$ gives $T_d < T$; $RH > 100$ gives $T_d > T$. The check reduces to "is reported RH above 100 %" — a range check.

The old flagship example — "dew point 33.1 °C exceeds air temperature 31.2 °C" — implies a reported **RH of 111 %**. A domain judge spots that immediately. Real cross-parameter physics is in the next section.

Correction 2 — the two-layer split is not the innovation. Every Layer 1 element is a named, standardised test in operational meteorological QC: gross-error/plausible-value, persistence/flat-line, step/temporal-consistency, internal consistency, and spatial-consistency (buddy) checks. An IMD reviewer recognises the whole suite on sight. It stays — as the credibility floor and the explainability substrate — but leading with it as novelty is the fastest way to lose the 25 %.

Correction 3 — health is a state, not an event. The earlier draft conflated "this reading is bad" with "this sensor is deteriorating". They are different objects with different lifetimes and need separate storage, separate update rules and separate UI.

Everything below is the revised plan. Where the reviews disagreed with each other, both positions are given.

The rubric is the plan

This statement publishes its own scoring weights. No other statement on the shortlist does. Every design decision below is chosen for what it earns.

CRITERION	WEIGHT	WHAT EARNS IT HERE
Innovation & Novelty	25%	A physical-signature classifier that names which fault or which weather event, not a rarity score

CRITERION	WEIGHT	WHAT EARNS IT HERE
Detection Accuracy	20%	Neighbour-difference channel + cross-channel discriminators; POD-vs-amplitude curves, not one F1
Real-Time Capability	15%	O(1) streaming state per station; published per-record latency and time-to-detection distributions
Explainability	10%	Uncertainty-propagated physics confidence; exact Shapley where a law was not broken
Scalability	10%	Per-station auto-calibrated thresholds – no manual tuning across a network
Practical Deployability	10%	Integer-arithmetic edge tier; measured bytes-per-station-day
Visualization / UI	5%	Three screens: board, judgement, planner
Energy Efficiency	5%	Uplink volume reduction, measured – radio TX dominates an AWS power budget

Innovation (25 %) plus Explainability (10 %) outweighs Detection Accuracy (20 %).
A classifier that names the fault and shows the physical reason scores on all three at once. A rarity score scores on one. That single observation should drive the whole architecture.

The one problem that actually matters

Everything hard compresses into one line of the statement's own text: *distinguish between genuine meteorological events and sensor/data anomalies.*

A real weather event looks statistically like an anomaly. A cold front produces a sharp pressure fall, a temperature fall and a humidity rise – three simultaneous multivariate excursions. A naive detector flags it. Flag enough real fronts and IMD stops trusting the system, which is the same as it not existing.

The discriminator is cross-channel coherence, not magnitude

Real phenomena move all three parameters in a physically consistent pattern. A failing sensor moves one. All four reviews converged independently on this being the strongest available signal, and it uses exactly the three parameters the statement allows.

PHENOMENON	JOINT SIGNATURE ACROSS T / P / RH	TIMESCALE
Thunderstorm gust front	Sharp T drop + RH jump + <i>pressure jump</i> (meso-high), all three together	minutes
Synoptic front passage	P falls smoothly, T follows, RH rises; neighbours see it with an advection lag	hours
Monsoon onset	Sustained RH rise + compression of the T diurnal range + modest P change	days
Heat wave	Sustained high T + low RH + persistent ridge; spatially broad	days
T sensor step	T only. Nothing else moves. Physically impossible for weather.	instant
RH element wetting	RH pins at 100 % while T climbs away from the dew point implied at pin onset	hours

This inverts the usual framing. Instead of *rarity* $\Rightarrow$ *anomaly*, the rule becomes **rarity + wrong physical signature $\Rightarrow$ fault**. A Mahalanobis score cannot make that distinction; a signature classifier can.

The physics you actually have with three parameters

With the tautology removed, here is what genuinely constrains or informs, and what each is good for. Note that only the first row is a hard impossibility – the rest are strong statistical-physical regularities, and they should be labelled as such rather than dressed up as laws.

RELATION	WHAT IT IS	CATCHES
RH > 100 % ($\equiv T_d > T$)	Hard impossibility – one check, honestly stated. Worth keeping, but as a range check with propagated uncertainty.	Gross RH failure, corrupt records
Dew point as an airmass property	T_d varies far less than T or RH over a diurnal cycle. A worked example below.	T or RH faults that break T_d smoothness
Specific humidity q	Conserved under dry adiabatic processes and diurnal heating. Requires all three parameters – this is what pressure is for.	Turns the 3-parameter limit into a design choice
T–RH anti-correlation	Rolling correlation of T and RH residuals is strongly negative at a healthy station; a stuck or drifting RH element collapses it toward zero.	RH degradation, O(1) streaming

RELATION	WHAT IT IS	CATCHES
Semidiurnal pressure tide (S2)	Twice-daily solar-locked oscillation, strongest and most stable in the tropics. Fit 2 diurnal + 2 semidiurnal harmonics and the residual becomes very tight.	Pressure faults; a genuinely distinctive result for India
Saturation onset divergence	At genuine saturation $T \approx T_d$ and T sits near the diurnal minimum. A wetted element stays pinned while T climbs away from $T_d(\text{onset})$.	Separates fog from a failed RH sensor
Spatial coherence with advection	Real fronts propagate — they reach a neighbour with a lag consistent with distance and plausible speed. Faults have no propagation structure.	The strongest fault-vs-event signal available

WHY DEW POINT AND q ARE THE USEFUL VARIABLES – WORKED

A single airmass through one day's heating. T swings 12 K and RH swings 41 points, but the two conserved-ish quantities barely move:

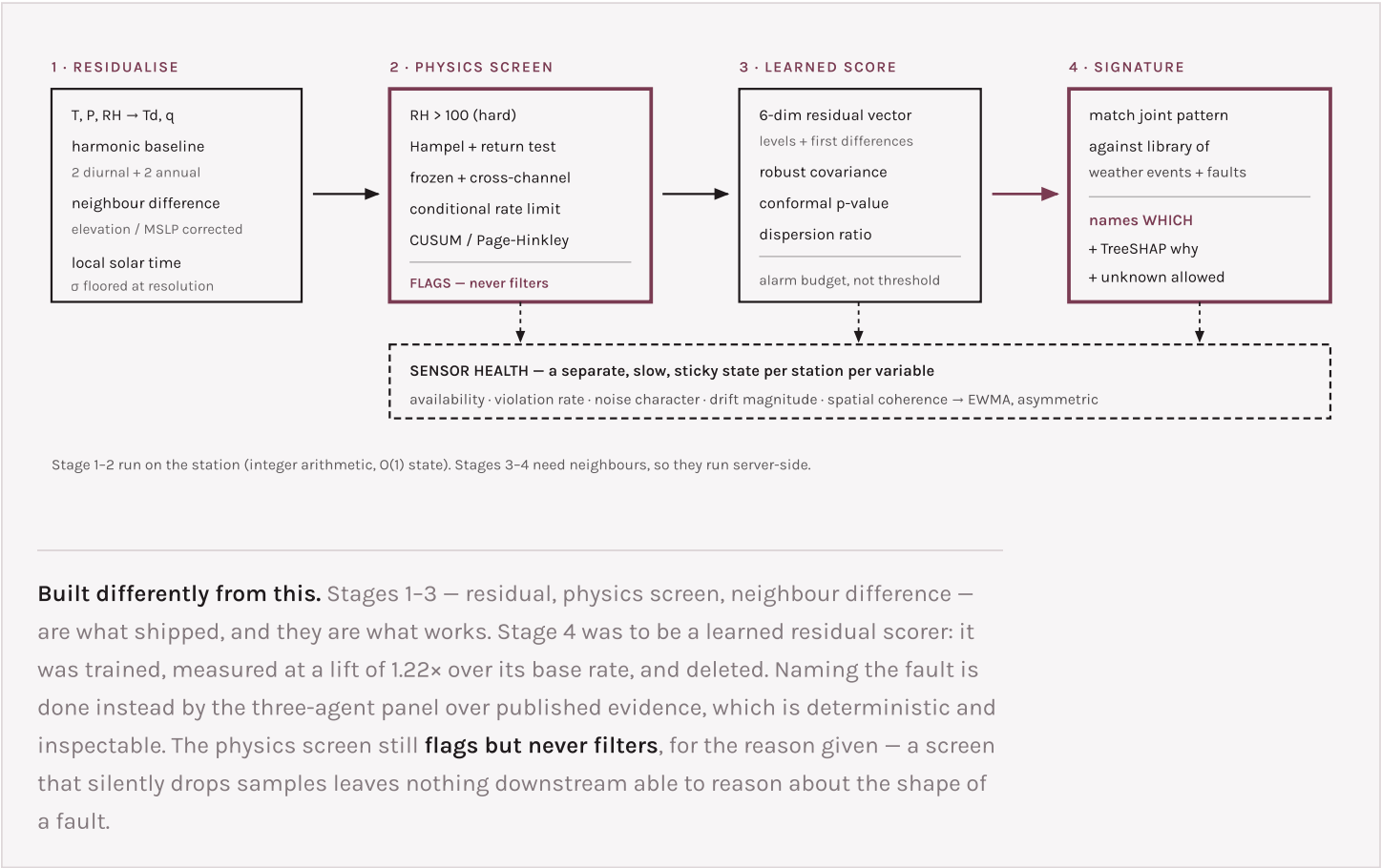
COMPUTED FROM MAGNUS + SPECIFIC HUMIDITY AT P = 1008 HPA

T (C)	RH (%)	Td (C)	q (g/kg)
22.0	88.0	19.92	14.48
28.0	64.0	20.55	15.07
34.0	47.0	21.09	15.59
30.0	57.0	20.55	15.07
24.0	79.0	20.13	14.67
swing: 12.0 K 41 pts 1.2 K 1.1 g/kg			

That contrast is the whole argument for residualising on T_d and q rather than on T and RH . A genuine airmass change moves them coherently and neighbours agree; a sensor fault breaks them while neighbours do not. **This is also the radiation-shield discriminator** — a failed shield or lost aspiration warms T and drops RH in a way that leaves T_d approximately correct, whereas real advection moves T_d .

Verify the empirical claims before you present them. The tide amplitude at your stations, the actual T_d stability at your sites, and the neighbour correlation length are all things to measure on real data — not to quote from this page.

Architecture



IDEA	WHY IT IS DEFENSIBLE	SERVES
Physics-propagated confidence	Confidence from instrument uncertainty, needing no <i>training labels</i> . See the explainability section.	Explain · Accuracy

TIER 2 – STRONG IF THE SCHEDULE HOLDS

IDEA	NOTE
Conformal alarm budget	Distribution-free false-alarm control with no fault labels. Lets the operator set an alarm budget instead of tuning thresholds. Cheap – quantiles over a ring buffer.
Physical self-calibration anchors	RH ceiling should sit near 100 % seasonally; $T_{\min}$ converges toward T_d on clear calm nights. Drift detection with <i>no neighbours needed</i> .
Noise-floor tracking	Degradation shows in the second moment before the first – σ rises before the mean drifts. Most teams watch only the mean.
Measured energy budget	Report bytes/station/day before and after. Radio TX dominates AWS power; a 90 % uplink reduction is the real energy result.

Novelty traps. No transformer or time-series foundation model – costly, weak on three variables, kills the edge story, and invites "why is this better than your median/MAD baseline?" No graph neural network over the station network – it needs a dense clean training set you do not have, and adaptive buddy checks beat it on sparse networks while staying explainable. No SHAP bolted onto the physics layer. And never claim ESP32 deployment without a measurement.

Detection, fault by fault

The single largest accuracy gain available is switching the primary detection channel from the raw residual to the **neighbour-difference series**, elevation-corrected. Weather cancels there; only the fault remains.

FAULT	DETECTOR	THE DISCRIMINATOR THAT MATTERS
Spike	Hampel filter, $k = 5\text{--}11$	Excursion and return . Without the return test, the first sample of a step is reported as a spike.
Frozen	Adaptive run-length	Are the other two channels still moving? Calm air flattens all three; a stuck sensor flattens one.

FAULT	DETECTOR	THE DISCRIMINATOR THAT MATTERS
Dropout	Gap + sentinel detection	Bit-identical across all three channels to full precision is essentially never natural → comms, not sensor.
Saturation	RH pin + divergence test	T climbing away from T_d (onset) while RH stays pinned. This is the fog discriminator .
Step offset	CUSUM on neighbour-difference	Level shift with diurnal shape and variance unchanged, and no recovery.
Drift	Page-Hinkley on daily-mean difference	Monotonically growing divergence from neighbours. Invisible locally, obvious spatially.
Noise burst	Rolling MAD-ratio / moving-range chart	Variance changes while the mean does not. Was entirely missing from the earlier plan.

ON DRIFT, HONESTLY

Sensor aging drift is of order 0.01–0.05 K/day. Weather variability is two to three orders of magnitude larger. No single-station statistic fixes that — the neighbour-difference series is the only thing that does, and it works because weather is spatially coherent while drift is not.

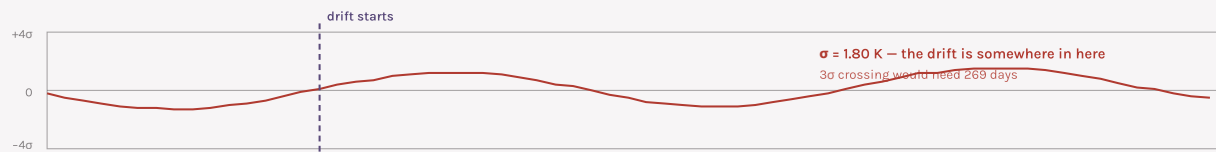
Detection latency scales as $\sigma_{\text{daily-difference}} / \text{slope}$. Measure your own σ on a clean period and publish the **latency-versus-slope curve**. If it comes out at weeks rather than days, say so. A team that reports "3–6 weeks at 0.02 K/day, here is the measured curve" is far more credible than one claiming real-time drift detection.

What you cannot do statistically: separate sensor drift from a genuine change in the station's environment — vegetation growth, a new building, a resited screen. They are identical in the data. Route it to a metadata and human-confirmation step, and say so on the slide. Judges will respect it; claiming otherwise is a hole they will find.

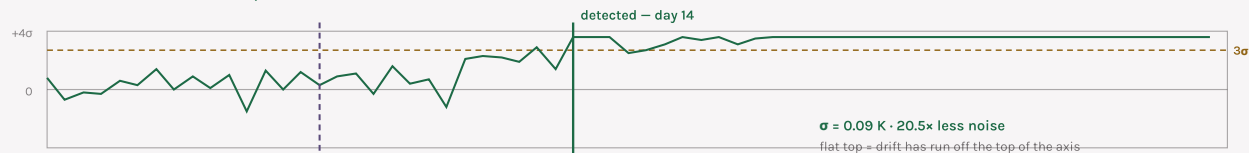
Where the accuracy actually comes from

A simulation of 180 days across three stations, with a **0.02 K/day** calibration drift injected into one of them on day 60. The same drift, viewed two ways:

SINGLE STATION – daily mean temperature deviation



NEIGHBOUR-DIFFERENCE – same data, same axis in σ units



That 20× gain came from a subtraction, not a model. No architecture choice on task A will buy you an improvement of this size – the residual engineering does. Spend the effort there first. *Caveat: in this simulation the neighbours share an identical seasonal signal by construction, so the real-world gain will be smaller. Measure your own σ – it is the number that sets your drift latency, and it belongs on a slide.*

The six accuracy traps

TRAP	WHAT GOES WRONG	DO INSTEAD
ROC-AUC	At sub-1 % prevalence it looks excellent for an operationally useless detector. Evaluators check for this.	PR-AUC
Point-level scoring only	A 30-day drift contributes ~720 true positives and inflates recall absurdly	Event-level too – the gap between them is informative
Event-level any-overlap alone	Measured trap. At 1 alert/station/week a <i>clean</i> 640-hour window already expects ~3.8 alerts, so P(at least one) is 0.98 with no fault present. Long events are credited for detection they did not earn.	Placebo events of matched duration – report real minus placebo
A single FPR	0.1 alarms/station/day overall but 8 during monsoon onset is unusable, and the aggregate hides it	Split by weather activity
Free-floating thresholds	Comparing methods at different alarm rates is meaningless	Fix the alert budget – 1/station/week – then compare recall
One accuracy number	Invites injecting only large obvious faults	POD vs amplitude curve; headline the minimum detectable amplitude at 90 % POD

The one-line summary. Residual engineering decides your accuracy. The fault classifier decides your innovation score. The injector decides whether you can prove either.

Explainability, as built

This section planned SHAP over a learned scorer. There is no learned model in the decision path, so that plan is void – and what replaced it is stronger, because it is exact rather than approximate.

SHAP is an approximation of Shapley.

It approximates only because a model has hundreds of features and enumerating 2^n coalitions is impossible. We have three agents. **2^3 is eight.** Every coalition is evaluated, so the panel's attribution is the exact Shapley value from the original theorem – three adjudications more expensive than the verdict itself.

- **The payout is escalation** – how hard a coalition pushes toward a visit. normal 0.0, warning 0.5, critical 1.0.
- **An absent agent is silenced**, returning `unknown` with no confidence, which is what the adjudicator already does with an agent that cannot see its evidence. Absence needed no special case.
- **The empty coalition scores zero**, so contributions sum *exactly* to the verdict – the efficiency axiom, and the property a force plot usually only claims.
- **Each agent ships a counterfactual**: the action the panel would reach without it. “Silence hardware and this becomes CALIBRATE” is the half an operator acts on.
- **The adjudicator records its walk** – every check, in order, with its answer, three-valued so a check whose evidence is missing reads “cannot tell” rather than “no”.

The gap, stated before anyone finds it.

The trace walks a *fixed* decision tree: across all sixty flagged rows on the board it takes two distinct paths from six question strings. It is an honest printout of the rules and it is not an autonomous investigation – nothing chooses what to examine next because of what it just found. That is designed and unbuilt.

What the operator sees

Lead with the verdict and confidence, then one sentence of reason. Always answer "why not weather" explicitly – that is the question an IMD operator actually has. Show percentages of the anomaly score, never raw Shapley units.

LAYER 1 · PHYSICS VIOLATION

STATION 42403 · Bhubaneswar · 09:15 IST · **INVALID** (p = 0.9994)

Reported RH 111.3 % – above the physical ceiling by 11.3 points, 4.2x the combined sensor uncertainty (+/- 2.7 %).

ROOT CAUSE RH sensor wet bias (0.71) · alt: calibration drift (0.19)

WHY NOT WX 0 of 3 neighbours show comparable humidity. Not a regional event.

COUNTERFACT No alert if RH < 102.7 % at this uncertainty.

ACTION Inspect/replace RH probe. T and P unaffected – continue using.

LAYER 2 · NO LAW BROKEN, SHAPLEY IS THE EXPLANATION

STATION 43128 · Nagpur · 14:40 IST · **ANOMALY** (p = 0.78)

No physical law violated. Residual 6.1 sigma from this station's normal 14:00 August behaviour. Conformal p = 0.0004 – at this threshold the expected false-alarm rate is 1 in 2,500.

DRIVERS pressure residual -3.8 hPa 61% of score
cross-channel inconsistency 25%
temperature residual +0.4 C 9%
humidity residual +2 % RH 5%

READING The pressure drop is not accompanied by the T and RH changes that normally accompany a drop this size here.

WHY NOT WX 4 of 4 neighbours show no pressure change, and no propagation lag consistent with a front.

ROOT CAUSE barometer step offset (0.64) · drift (0.24) · unknown (0.12)

Two non-negotiables: emit **unknown** rather than forcing a label – a confidently wrong root cause destroys operator trust faster than an honest "cause unclassified"; and **version-stamp detector, explainer and calibrator on every alert**. An explanation that cannot be reproduced later is not an audit trail, and a met agency will ask.

Sensor health is a state, not an event

Health is not "is this reading bad". It is a slowly-moving estimate of **how much to trust this sensor's future readings**. Keep two separate objects:

OBJECT	GRAIN	LIFETIME
observation_flags	Per timestamp, per variable	Ephemeral — an event
sensor_health	Per station, per variable + a station roll-up	Persistent, updated on a schedule — a state

Per-variable is essential: a station with a dead RH element and a perfect barometer must not report as "50 % healthy". **The station roll-up is the minimum of the three, not the mean.**

FIVE COMPONENTS

COMPONENT	WEIGHT	WHAT IT MEASURES
Physical-plausibility rate	0.25	Rate of hard violations over trailing 7 days, exponentially weighted. Most trustworthy input, so heaviest.
Bias / drift magnitude	0.25	Current estimated offset as a fraction of instrument spec tolerance. The "deteriorating" axis.
Availability	0.20	Fraction of expected observations received, plus gap burstiness. Telemetry loss is the most common real AWS problem.
Noise character	0.20	Penalise $ \log(\sigma_{\text{recent}} / \sigma_{\text{baseline}}) $ — both directions . Variance collapse precedes stiction; inflation means a loose connection.
Spatial coherence	0.10	Correlation with neighbour consensus over 30 days. Low weight — confounded by genuine microclimate at hill and coastal sites.

FIVE STATES, WITH STICKY TRANSITIONS

STATE	BAND	MEANING	ACTION
HEALTHY	H ≥ 85	Trust the data	none
WATCH	70–85	Off baseline, not yet actionable	system watches
DEGRADING	50–70	Persistent drift; data usable with correction	schedule maintenance
SUSPECT	25–50	Should not be used unqualified	correct or withhold
FAILED / SILENT	< 25	Unusable, or no data > 6 h	dispatch

Make the EWMA asymmetric. Degradation must register faster than recovery — use a larger α when health is falling. A sensor that looks fine for two hours after a week of violations has not been repaired. Only an explicitly logged calibration event should reset health quickly, and making that an operator action in the UI is a good demo beat. Add hysteresis so stations do not flap on a band boundary.

Distinguish SILENT from FAILED visually — hollow ring versus filled dot. The remedy differs: comms/power versus sensor.

THREE AXES THAT SEPARATE THE THREE CASES

	BAD READING	DETERIORATING SENSOR	GENUINE WEATHER
Duration	seconds-minutes	days-weeks, monotone	minutes-hours
Neighbours	disagree	disagree, and the gap grows	agree, with advection lag
Recovery	returns to baseline	never returns	returns along a physical trajectory
Cross-variable	usually one variable	usually one variable	coherent across T/P/RH

Degradation and maintenance

Fit the neighbour-difference residual with robust regression — **Huber or Theil-Sen, mandatory**, because outliers from genuine faults would otherwise dominate the slope — over a trailing 60-90 days. Then:

```
TTOS = ( spec_limit - |b0_now| ) / |b1|      report the interval, always
```

Present as " ≈ 47 days (18-140)", never a bare number. A wide interval is information.

Do not attempt a Weibull / survival RUL model. You have no failure-history dataset. Any hazard curve you fit would be fabricated. Linear extrapolation of measured drift with honest confidence bounds is defensible in front of a domain judge; an unfittable Weibull is not — and saying so out loud reads as maturity.

Also: **TTOS predicts specification excursion, not failure.** Label it exactly that way in the UI. Most real degradation is abrupt, not linear.

Add a second, non-drift trigger: **variance-collapse time**. When the noise component falls monotonically for N days it often precedes stiction failures, which pure drift extrapolation misses entirely.

Sort the work queue by a **maintenance priority index**, not by TTOS alone:

```
priority = urgency x criticality x (1 / accessibility_cost)
```

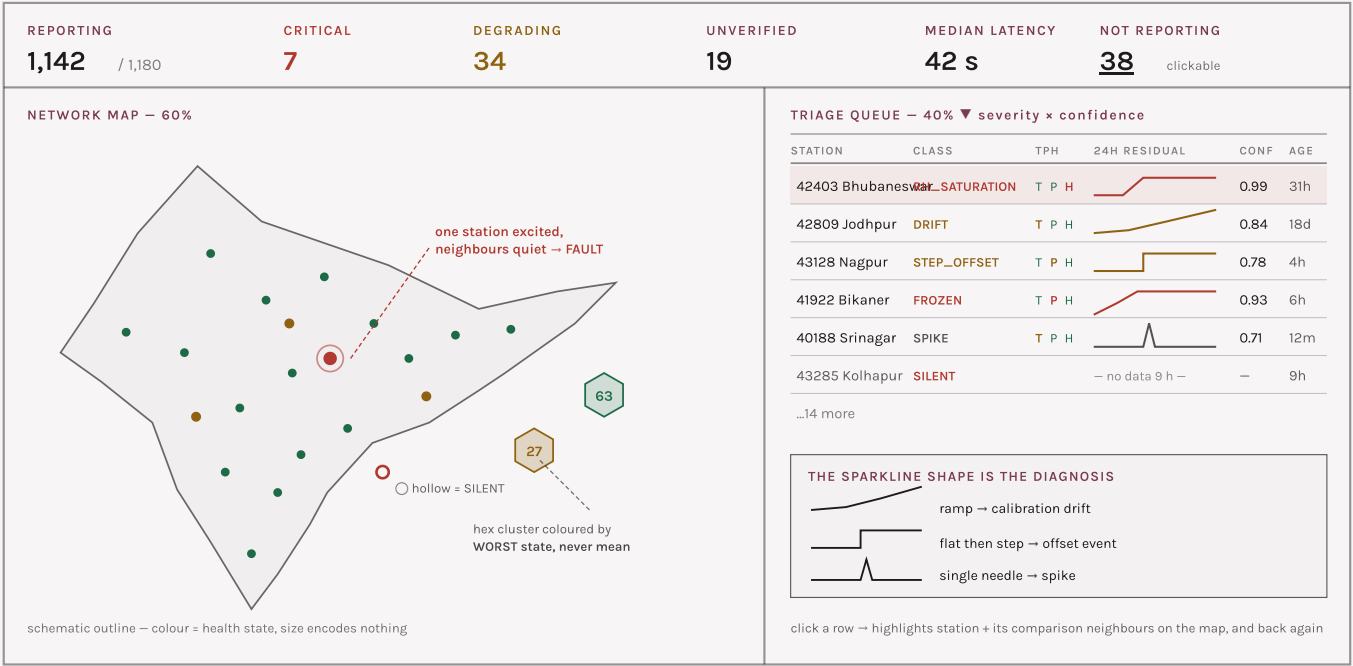
Criticality is a hand-populated config field — aviation, flood-warning basin, model-assimilation station, ordinary. And add a "**nearby also due**" indicator: a real maintenance engineer's constraint is travel, not sensors, and batching a field trip is the feature that makes their eyes light up.

Honest limits: if a whole region drifts together — a bad firmware version, a supplier batch — your neighbour reference drifts with it and you see nothing. Mitigate by also regressing against long-term climatology and by monitoring the network-wide mean residual as its own signal. Pressure drift is the most detectable; **RH is the hardest** and degrades non-linearly at the high end. Below ~30 days of history, show "*insufficient history*" rather than a number.

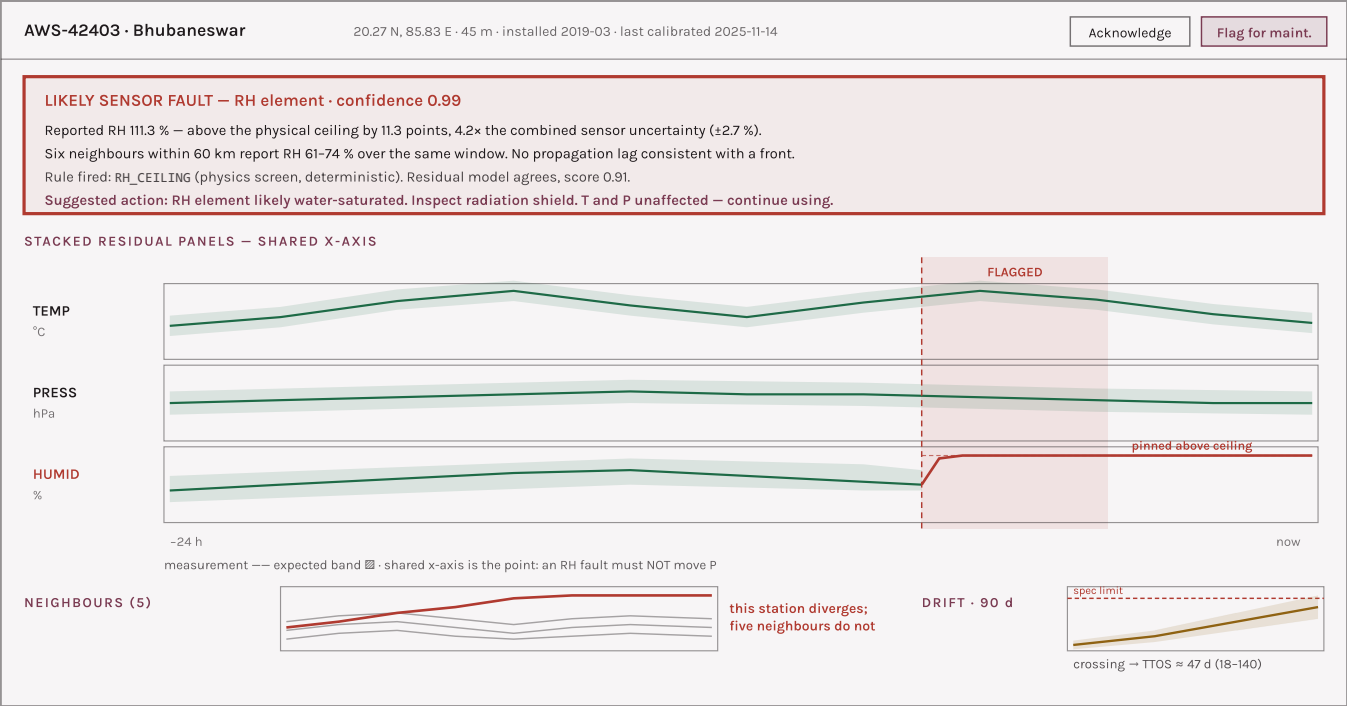
The dashboard superseded

These wireframes are design history. The built interface is four routes, drawn in “What the operator sees” above — the shape held, the detail did not.

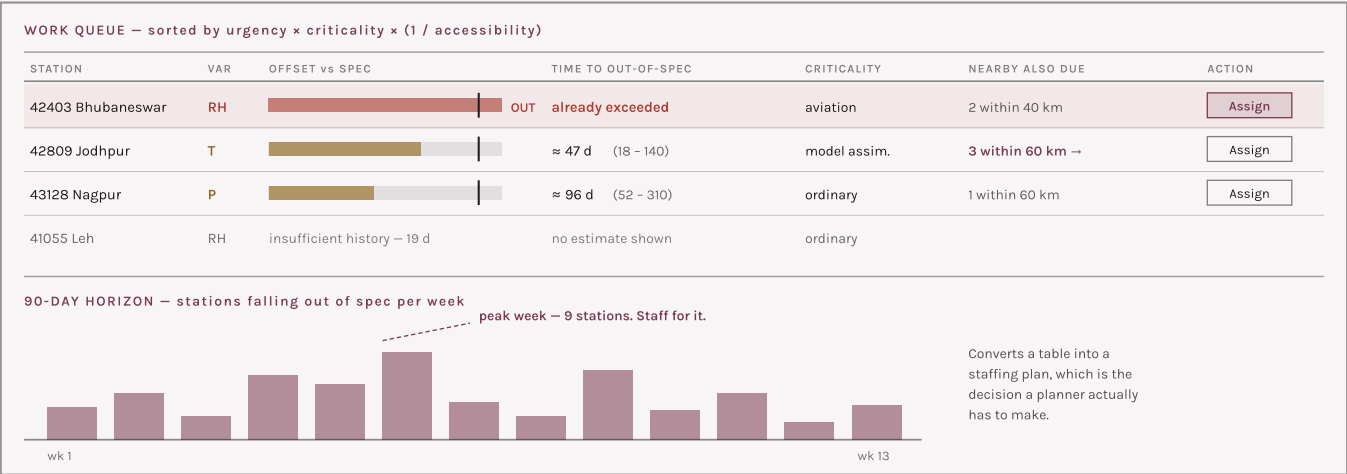
An IMD engineer's day is triage, not exploration. But the moment they open a suspect station they need exactly one very good chart, because that is where the weather-versus-fault judgement actually gets made by a human. The rule: **every visual must answer a question the operator is about to act on.**



Screen 1 — the network board. The map earns its place because spatial correlation is the discriminator: one dot excited while its neighbours stay quiet reads as a fault before you have consciously thought about it. Rows are ~28 px so twenty fit without scrolling, and the sparkline shape carries most of the diagnosis on its own.



Screen 2 — the judgement screen. The verdict card does the explainability work in plain sentences before any chart appears. The three panels share an x-axis deliberately: a genuine front moves all three together, so the fact that pressure and temperature stay flat while humidity pins is itself the root-cause evidence — visible only because they are stacked.



Screen 3 — the maintenance planner. Sorted by priority rather than by time-to-out-of-spec, because a distant failure at an aviation station outranks an imminent one at an ordinary site. The *nearby also due* column exists because a field engineer's real constraint is travel, not sensors — and note the fourth row showing *insufficient history* rather than inventing a number.

Top strip: five large counters — reporting 1,142 / 1,180, critical, degrading, unverified alerts, median latency. Make "not reporting" clickable; silence is the most common real fault and most dashboards forget it.

Left 60 % — map. Stations as small circles, colour = health state, size encodes nothing. Above a station count, cluster into hex bins coloured by the **worst** state in the bin, never the average — an operator must never have one critical station averaged into a green hex.

The map earns its place because *spatial correlation is the discriminator*. Eleven adjacent stations showing a pressure drop is a system moving through; one showing it is a barometer. The map makes that inference pre-attentive.

Right 40 % — triage queue. Dense table sorted by severity × confidence. Row height ~28 px so twenty rows are visible. Columns: station, fault class, a tiny coloured T/P/RH triad, a **24 h residual sparkline**, severity bar, confidence, age, state chip.

The sparkline is the workhorse. The shape of the residual is almost fully diagnostic — flat then a step is a calibration event; a ramp is drift; dead-flat at non-zero is frozen; a single needle is a spike. An operator learns to read these in under a second. That is where information density comes from.

Wire the map and queue **bidirectionally** — clicking a row highlights the station and its comparison neighbours; hovering a station scrolls the queue. Thirty lines of code, and it is what makes the demo feel like an operational system rather than two widgets side by side.

SCREEN 2 · STATION DETAIL — THE JUDGEMENT SCREEN

A **verdict card** in plain sentences at the top (the format shown in the previous section), then the main chart: three vertically stacked, **x-axis-linked** panels for T, P, RH. Each shows the raw measurement, the expected band as a translucent ribbon, and flagged intervals as coloured vertical bands behind everything.

Shared time is the entire point — faults in one variable print in another. An RH failure will not move pressure; an enclosure or power problem moves all three together. That co-movement is itself the root-cause signal, and the stacked layout is what makes it visible. Below: a neighbour-comparison strip, then the drift panel with its spec-limit line, then an event log.

The prioritised work queue, grouped by region for trip batching, plus a 90-day horizon strip showing how many stations fall out of spec per week — which converts a table into a staffing plan.

Hackathon dashboard failure modes: the chart gallery with nothing to act on; demoing on three stations (kills scalability credibility instantly); a "Refresh" button instead of live updates; everything red because thresholds are untuned; no empty, loading or error states; beautiful but decision-free; a model score with no explanation; slow first paint burning a minute of a six-minute demo. Build and test at projector resolution, not laptop.

Data strategy and injection realism

The statement removes the usual blocker: "*Participants may use historical AWS datasets, simulated anomalies, or streaming sensor data*", evaluated on "*anomaly injected data*". Synthetic faults are the sanctioned methodology.

Use **real station history for realism** (public hourly surface observations give genuine diurnal and seasonal structure, real noise, and real events to *avoid flagging*) and **injected faults for evaluation**, where you control ground truth.

THE SELF-FLATTERY TRAP, AND HOW TO STRUCTURALLY AVOID IT

The dominant failure is injecting only large, obvious faults. Counter it by design:

- **Sweep magnitude and report a curve, not a number.** Probability of detection versus anomaly amplitude in units of local σ , at a fixed alert budget. Headline the **minimum detectable amplitude at 90 % POD**. This single change does more for credibility than anything else here.
- **Include sub-threshold magnitudes.** Spikes 2–20 σ . Steps down to the noise floor. Drift 0.005–0.1 K/day over 10–180 days. Frozen 30 min to several days. Noise $\times 2$ to $\times 20$.
- **Hard cases deliberately.** Freeze at a value *plausible for that hour*. Pin RH at 99.9 or 100.5, not exactly 100. Inject a step whose sign matches the prevailing trend.
- **Onset shapes:** instantaneous, linear ramp, exponential-asymptote (typical of aging), and **intermittent/flapping** — a sensor failing for 20 minutes every few hours is extremely common in the field and almost always missed by systems tuned on clean single-episode injections.
- **Inject during real weather events**, not only quiet periods. A spike on top of a thunderstorm is what separates a good system from a demo.
- **Multi-fault:** drift plus occasional spikes — the realistic aging picture.

- **Negative controls:** matched clean windows, plus a distractor set of real events with no injection.

Evaluation protocol

Build this *before* the model. Accuracy is 20 %, and the only way to defend a number is to have measured it the same way since day one.

METRIC	WHY THIS ONE
Per-fault-type table	Aggregate F1 is dominated by whichever class is numerous and easy, and completely hides drift
PR-AUC, not ROC-AUC	At realistic prevalence (well under 1 %) ROC-AUC looks excellent for an operationally unusable detector. Evaluators look for this trap.
Alerts per station per day	Precision is prevalence-dependent and your injected prevalence is not the real one. This framing is prevalence-free.
FPR split by weather activity	0.1 FP/station/day overall but 8 during monsoon onset is useless — and an aggregate hides it completely
Event-level and point-level	Point-level lets one long drift contribute thousands of true positives and inflate recall absurdly. The gap between the two is itself informative.
Time-to-detection distribution	Median and 90th percentile per fault type. This is your Real-Time evidence (15 %).
Calibration error	Reliability diagram, ECE, Brier. Confidence is a scored output, so it must mean something.
Throughput	Stations/second on one core. Scalability as a number.

Fix the alert budget across all compared methods. Tune every detector to, say, one alert per station per week on validation, then compare recall. Otherwise the comparison is meaningless.

Split by whole stations and whole time periods, never random points — adjacent samples are near-identical and random splits leak.

Event-level recall has its own duration bias — measured

Point-level scoring inflates recall on long faults, so the obvious fix is event-level any-overlap: the event counts as detected if the detector fires even once inside it. **That fix has a bias of its own, in the opposite direction, and it is large.** At one alert per station per week,

a *clean* 640-hour window already expects ~3.8 alerts — P(at least one) is 0.98 whether or not a fault is there.

The control is a **placebo event**: same station, same duration, placed on a window with no injected fault at all. Recall on placebos is the floor the metric hands you for free. Report the **excess** — real minus placebo.

EVENT DURATION	TEMP REAL	TEMP PLACEBO	TEMP EARNED	PRES EARNED	RH EARNED
< 6 h (spikes)	0.50	0.006	0.494	0.667	0.000
6–24 h	1.00	0.058	0.942	0.979	−0.042
1–4 d	0.83	0.149	0.684	0.723	0.677
4–16 d	0.67	0.386	0.281	0.447	−0.012
> 16 d	1.00	0.632	0.368	0.365	−0.632

1 alert/station/week, 480 placebo events per variable, from `evaluation/run_placebo.py` .
Pressure at 6–24 h is the strongest cell measured: 1.000 real against 0.021 placebo.

Short faults are earned almost entirely. Long ones are not — `calibration_drift` recall 1.00 beyond 16 days is roughly a third earned; the rest is event length.

Relative humidity fails outright beyond a day. At >16 d it earns **−0.632**: real recall 0.000 against a placebo floor of 0.632, so the pipeline detects *none* of the long humidity faults while random windows of the same length fire 63 % of the time. Reported rather than dropped, because a metric that cannot fail is not measuring anything — and because the RH channel plainly needs work before it is quoted anywhere.

Rule: never quote an event-level recall without the placebo floor beside it.

Traps that will bite you

Every item here was raised as a concrete failure mode, several of them fatal. Read this section before writing code, not after.

TRAP	CONSEQUENCE	FIX
Online baseline updating	Slow drift is absorbed into the baseline and becomes permanently invisible. The most	Fit climatology on a frozen, verified-clean reference period. Refit only on human-

TRAP	CONSEQUENCE	FIX
	dangerous item on this list.	approved data.
MAD = 0 on quantized data	Infinite z-score. Floods false spikes on the calmest, most benign nights.	$\sigma^{\wedge} = \max(1.4826 \cdot \text{MAD}, c \cdot \text{resolution})$, $c \approx 0.5\text{--}1.0$
Raw pressure neighbour comparison	~1 hPa per 8–9 m elevation. Two stations 100 m apart differ by ~12 hPa with zero anomaly — every hill station looks broken.	Reduce to MSLP or a common level. <i>Confirm whether your feed carries station pressure or MSLP.</i>
Hour-of-day in UTC or IST	India spans ~29° of longitude — solar noon differs by nearly two hours end to end. Smears every diurnal baseline.	Use local solar time, referenced to longitude
Fixed rate limits	Fire constantly in the pre-monsoon convective season, when downdrafts produce very large short-timescale T drops	99.9th percentile of $ \Delta x $ per station-month-hour, floored by a hard physical bound. Apply to the neighbour-difference where possible.
Night-time T neighbour comparison	Nocturnal decoupling and cold-air pooling make inter-station differences large and non-stationary. Top false-alarm source.	Hour-conditional thresholds. Select neighbours by lowest σ of the difference series, not by distance.
Physics layer filtering	Rejected samples never reach the learned layer, which can then never reason about fault shape	Flag, never filter
Sentinel values	<code>-999</code> , <code>-9999</code> , <code>0.0</code> parsed as real readings poison every baseline. The most common data bug in this problem class.	Audit ingest now, before anything else
MCD on contaminated data	Masking — the covariance absorbs the anomalies and then cannot see them	Fit on verified-clean data only
CUSUM ARL assumptions	ARL guarantees assume independence. Meteorological residuals are strongly autocorrelated, so your nominal false-alarm rate is not the real one.	Whiten first (AR(1) innovations) or set h empirically to your alert budget
Centred-window detectors	A centred Hampel window costs half-window latency — trivial at 1-minute data, unacceptable at hourly	Decide causal vs centred per detector and publish the latency each costs

Stack and repository layout

LAYER	CHOICE	NOTE
Core	Python 3.11, numpy, pandas, scipy	CPU only – no GPU anywhere
Models	scikit-learn, statsmodels RLM, SHAP	Not built. There is no model in the decision path; attribution is exact Shapley over the three agents, which needs no library
API	FastAPI + WebSocket	Push deltas only – never re-send the whole network
Storage	SQLite (WAL)	One file. TimescaleDB is the production design, not what runs – see <i>Production architecture</i>
Current-state cache	Redis	Not built. One process, in-memory
Map	Leaflet	MapLibre was the plan; Leaflet is what shipped. Same argument holds – no API key, no billing
Charts	Hand-rolled inline SVG	No chart library at all. uPlot was never needed at this data volume
Edge tier	ESP32 / Arduino C++	Built. Level-0 screen and housekeeping on the node, TLS uplink, verified in Wokwi

```
skyguard/  
├─ physics/           Stage 2 – pure functions, zero dependencies  
│   ├── relations.py   Magnus (water/ice), q, Td, vapour pressure  
│   ├── checks.py      range, Hampel, frozen, conditional rate  
│   ├── uncertainty.py instrument specs → propagated confidence  
│   └─ test_physics.py property tests  
├─ residual/          Stage 1 – harmonic baseline + neighbour difference  
│   ├── harmonics.py   2 diurnal + 2 annual, robust fit  
│   ├── neighbours.py  selection by lowest difference-sigma, MSLP reduction  
│   └─ solartime.py     longitude-referenced local solar time  
├─ detect/            Stage 3 – CUSUM, Page-Hinkley, dispersion, Mahalanobis  
├─ signature/          Stage 4 – the fault + event classifier (the novelty)  
├─ health/            sensor_health state machine, TTOS  
├─ explain/           exact Shapley, grouped attribution, counterfactuals  
├─ inject/            fault generator with ground truth  
├─ evaluation/        the harness – build before any model  
├─ api/              FastAPI, WebSocket, ingest, the three-agent panel  
├─ web/              React + Vite + Leaflet, four routes  
├─ dashboard/        the JSON exports the API serves  
├─ firmware/         ESP32, mirrors physics/  
└─ data/             station history, cached (gitignored)
```

physics/ has no dependencies on purpose. Pure arithmetic over three floats — unit-testable in Python, transliterable to C++ for the ESP32, and citable in the report as the deterministic tier. Keep pandas out of it and it ports in an afternoon.

Production architecture design only

None of this is built. Storage today is a single SQLite file in WAL mode and there is one process; TimescaleDB, Redis and a worker pool are what a deployment at IMD scale would need, not what runs.

The prototype and the deployed system are different animals, and a judge asking "*how does this actually get used at IMD?*" is probing for this section. It opens with arithmetic, because the arithmetic is on your side.

RUN THE SCALE NUMBERS BEFORE DESIGNING ANYTHING

IMD operates on the order of **2,000 AWS stations** (*verify the current figure*). At 15-minute reporting:

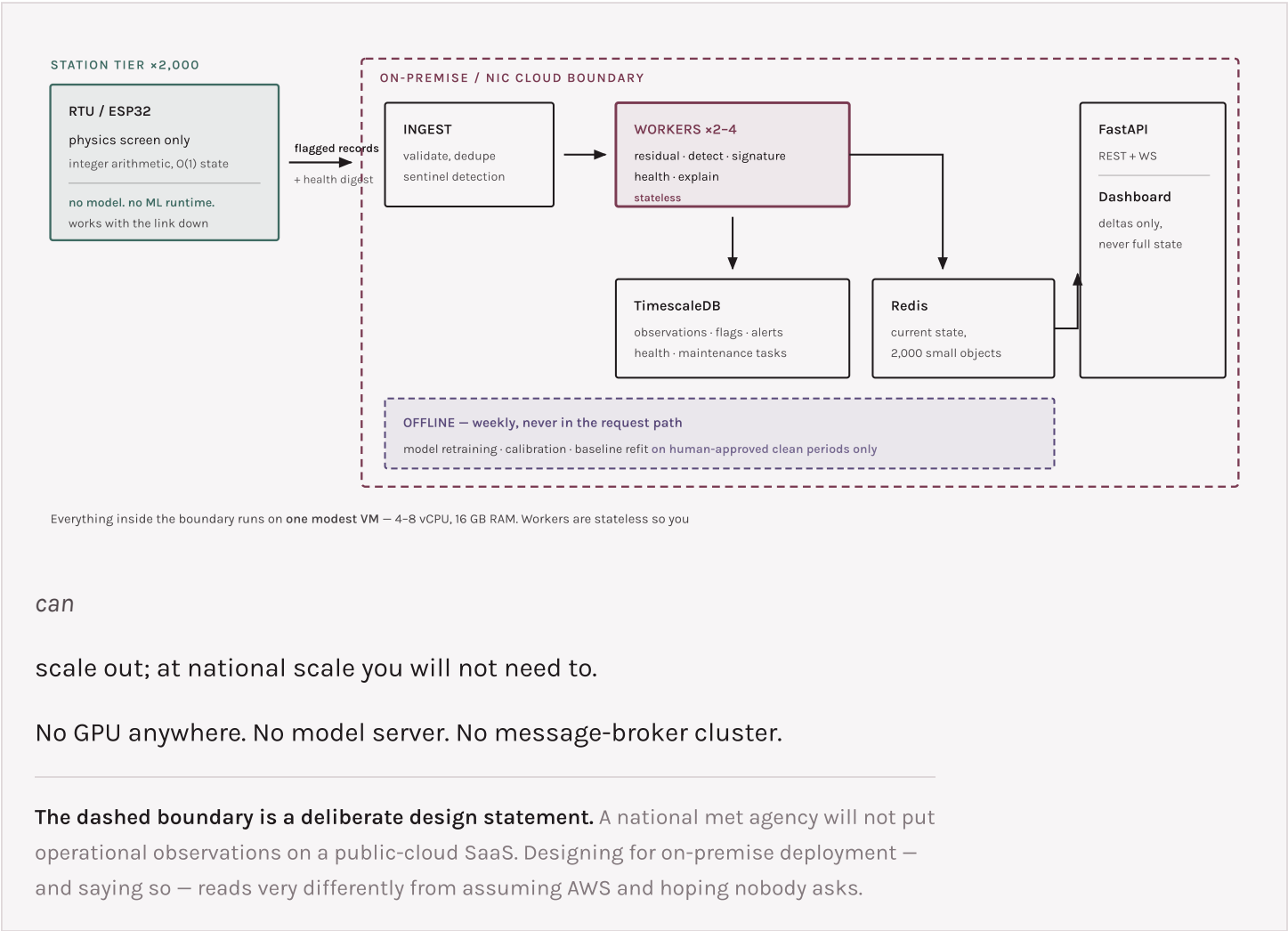
NATIONAL NETWORK, COMPUTED

stations	2,000
records / day	192,000
records / year	70,080,000
raw bytes / year	3.50 GB
compressed (~10x)	350 MB
10 years compressed	3.50 GB
average ingest rate	2.22 rec/s
quarter-hour burst	2,000 records
burst clear @ 200 us/rec, 1 core	0.40 s

This is not a big-data problem. A decade of national AWS observations fits on a laptop SSD, and the quarter-hourly burst clears in under half a second on a single core. Anyone proposing Kafka, Spark and a cluster is overengineering it — and a domain judge will know. **Say the numbers out loud.** Restraint reads as competence here; a distributed diagram reads as inexperience.

Budget those per-record timings, then measure them. Do not quote them from this page.

THE DEPLOYED TOPOLOGY



DATABASE — POSTGRESQL + TIMESCALEDB

REQUIREMENT	WHY TIMESCALE
Time-series partitioning	Hypertables handle it without you writing partition management
Chart rollups	Continuous aggregates maintain the hourly/daily views the dashboard needs
Storage	~10× compression on observation data — 3.5 GB for a decade nationally
Joins	You need them. Alerts → stations → health → maintenance tasks. A pure time-series database fights you here
Deployability	Open source, installs inside a government network, no vendor account

Why not the alternatives: InfluxDB is weak relationally and its licensing has shifted; ClickHouse is excellent analytically but you also need transactional alert and

acknowledgement state; plain PostgreSQL is genuinely fine and Timescale mainly saves you writing rollup logic yourself.

WHERE THE ML ACTUALLY RUNS

TIER	RUNS	WHY THERE
Station	Physics screen only	Instant hard-fault detection, survives a dead link, cuts uplink volume. No model, no ML runtime — it is arithmetic.
Server, online	Residual · detectors · signature · health · explain	Needs neighbour data, which a station does not have
Server, offline	Training · calibration · baseline refit	Weekly. Never in the request path , and baselines only on approved clean periods

Inference does not need a model server. LightGBM predicting on 12 features is microseconds. Load the model into the worker process at startup as a versioned artifact. No TF Serving, no Triton, no separate inference service — adding one is a liability you would then have to defend, for zero gain.

MIGRATION PATH – THE CODE DOES NOT CHANGE

Keep the data-access layer thin: plain SQL or SQLAlchemy Core, which SQLite and PostgreSQL both speak. Then the prototype becomes production by swapping implementations behind the same interfaces.

	PROTOTYPE (NOW)	PRODUCTION (LATER)
Database	SQLite file (or DuckDB)	TimescaleDB
Current state	Python dict in-process	Redis
Process model	One <code>uvicorn</code> , serving the React build	Ingest + workers + API
Deploy	<code>docker compose up</code>	Same compose file, more services
Demo	Local laptop, zero network	On-premise VM

Do not build the production topology now. Two people building ingest queues, worker pools and a Timescale cluster will spend the entire timeline on infrastructure and arrive with nothing to demonstrate. Build the single-process prototype against interfaces; present the production topology as the design.

And for the finals demo itself: **run locally.** Venue Wi-Fi failing mid-demo is a documented way to lose. Hosting is for teammates and pre-round submission, not for the room.

THE LINE TO SAY OUT LOUD

"Two thousand stations at fifteen-minute cadence is 70 million records a year and 2.2 records a second. The entire national network's real-time quality control fits in a fraction of one CPU core, on one on-premise VM, with no GPU anywhere. We chose methods that make that true rather than methods that would need a cluster."

That answers **Scalability (10 %), Deployability (10 %) and Energy Efficiency (5 %) in one breath** — and it is stronger than any architecture diagram, because it is measured rather than asserted.

Success criteria

MEASURE	TARGET	SERVES
Hard-fault recall (spike, frozen, dropout, saturation)	> 98%	Accuracy
Minimum detectable step at 90% POD	< $1.0 \sigma_{\text{local}}$	Accuracy
Drift detection latency at 0.02 K/day	report measured curve	Accuracy · Real-time
Real weather events not flagged	> 99%	Accuracy
Alerts/station/day during <i>active weather</i>	< 0.5	Accuracy
Signature classification accuracy (7 classes + unknown)	> 85%	Innovation
Confidence calibration error (ECE)	< 0.05	Explainability
Latency per reading, full pipeline	< 50 ms	Real-time
Throughput, single core	> 1,000 stations/s	Scalability
Uplink bytes/station/day, before vs after	> 90% reduction	Energy

Targets, not results. Replace each with harness output before it goes in front of a judge.

Hardware runs in simulation

A **software** category statement, so pure software is fully compliant. The node exists anyway: `firmware/skyguard_node` runs end to end in Wokwi, posting over TLS to the deployed service through the same ingest door a physical board would use. What follows is the bill of materials for building one for real, which has not been done.

ITEM	MEASURES	₹
ESP32 dev board	—	600
BME280	Temperature, pressure, humidity — <i>exactly the three named parameters</i>	250
DS18B20	Independent temperature reference	120
Breadboard, jumpers	—	150
Per station		1,120

Build two, ₹2,240. That demonstrates the spatial check from IMD's own example — one station lying while its neighbour reads normally. Two *different* temperature sensors matter: identical parts share failure modes, so if both drift together you learn nothing.

The demo that lands: breathe on the BME280 and watch RH climb toward saturation while the node distinguishes it from a wetted element — because T is tracking T_d coherently — then compare against the second station reading normally one metre away.

Deliverables

The statement's final line is a hard requirement that is easy to overlook:

"Fully executable code with example usage and a document explaining various use cases."

Not a prototype and slides. **Runnable code** with worked examples, plus a use-case document. Budget for it: a clean README with a one-command demo, example notebooks per fault type, and a use-case document covering the operator workflows — triage, judgement, maintenance planning. Write the document as you build, not in the last week.

Build log — what exists, what was measured, what failed

Superseded in part. The drift headline below — “recall 1.00 at 0.01 K/day” — was measured with any-overlap event scoring, which credits long events for detection they did not earn: at one alert per station-week a clean 640-hour window already expects ~3.8 alerts. Against a matched-duration placebo control the drift class earns roughly a third of that, not 1.00. The naming figure of 0.706 was also measured before a split bug that left 37.6 % of test rows with a NaN primary channel; it is 0.611 once fixed. Read **Measured so far** below for the corrected figures — where the two disagree, that section wins.

Everything above this line is design. Everything below it is **measured on real ERA5 data for ten stations across Assam and Maharashtra**, degraded to AWS realism, with faults injected and ground truth attached. Numbers are event recall at **one alert per station per week** on held-out stations and held-out months, unless stated otherwise. Where a number moved because a bug was fixed rather than because the system improved, it says so.

Built and running

STAGE	WHAT IT DOES	STATUS
Data	ERA5 audit, AWS-realism degradation, data-derived neighbour graph	Zero missing values found; two of nine columns were exactly derived and dropped
Harness	Seven fault types injected with point- and event-level truth; PR-AUC, POD-vs-amplitude, FA split by weather activity, time-to-detection	458 events, ~3 % prevalence, ten independent replicates
1 · Residualise	Magnus relations, local solar time, MSL reduction, harmonic baselines by Huber IRLS	30 baselines, 270 coefficients, fitted on a frozen window with no labels
2 · Physics	Neighbour-difference z, run-length persistence, dropout assertion	Causal — trailing window only
3 · Learned	Robust covariance (MCD concentration), normal-score marginals, split-conformal p-values	One detector per variable, six axes each
4 · Signature	Nine templates over eight scale-free descriptors, names the fault or abstains	0.706 naming accuracy, 19 % honest unknown
Health	Sticky per-probe state with hysteresis, worst-channel roll-up	Separate from per-timestamp flags, by design

STAGE	WHAT IT DOES	STATUS
Service	FastAPI, replay clock, tile proxy with disk cache, OpenAPI docs	Whole pipeline fitted in ~10 s at startup; budget is a live dial
Console	Live map, neighbour edges, residual colouring, replay, alert inspector	Falls back to a static build with no server

The headline result

Calibration drift is detected at 0.01 K/day with recall 1.00, at one alert per station per week. That is about a third of a degree per month – a drift no operator can see by eye, that silently corrupts a climate record, and that a single-station detector cannot find at all. Pressure and humidity reach recall 1.00 at 3.5 alerts per station-week. This is what the neighbour-difference channel was built for and it is the number the submission should lead with, ahead of any accuracy percentage.

What is amplitude-limited, and what is budget-limited

Spike recall came out identical to two decimal places for five different detectors – a plain z-score, a Mahalanobis distance, a Hampel filter written specifically for spikes, and two ensembles. When methods that share no mathematics agree exactly, the number is not describing any of them. It describes the injected amplitude against the noise floor.

FAULT CLASS	LIMIT	EVIDENCE
calibration_drift	Solved	recall 1.00 at 0.01 K/day, temperature
frozen, dropout, saturation	Solved	recall 1.00 – structural, not statistical
step_offset	Budget-limited	humidity 0.42 → 1.00 as budget goes 1 → 21 alerts/week
noise_burst	Budget-limited	humidity 0.29 → 0.86; pressure 0.50 → 1.00
spike	Amplitude-limited	POD 0.00 below 1.65σ; more budget moves 0.24 → 0.48 and stops

The honest figure for spikes is therefore a **minimum detectable amplitude**, not a recall percentage. “We detect a 3 K spike; a 1 K spike is below the noise floor at this alert budget” is a claim an evaluator can check. A recall number here would be a statement about our own injector.

Tried, measured, rejected

These are kept in the repository with their numbers attached. Three negative results with measurements behind them are a stronger position than three untested ideas — they are the difference between “we did not try that” and “we tried it, here is what it cost”.

IDEA	RESULT	WHY IT FAILED
Supervised detection (GBT)	No consistent win; hurt pressure as a fourth channel	Leave-one-fault-type-out gap +0.161 . Withhold <code>calibration_drift</code> and pressure recall goes 1.00 → 0.00 — it was recognising the injector's linear ramp, not drift
Supervised naming	0.500 vs templates' 0.710	Never abstains. Withhold a class and it is confidently wrong on 100 % of <code>calibration_drift</code> , <code>dropout</code> and <code>frozen</code> windows — a softmax has no way to say “I have not seen this”
Hampel + dispersion channels	No change at all	Spike recall identical across all five detectors — the limit is amplitude, not method
Per-fault-type budgets	Equal spend, recall slightly worse	Of 8,102 candidate windows the classifier labelled 4,717 <code>genuine_weather</code> , 1,960 <code>unknown</code> , 5 <code>calibration_drift</code> and 0 <code>noise_burst</code> . You cannot budget by class when class membership is wrong
Simes / Fisher combination	0.436 / 0.270 vs 0.450 for minimum-p	Fisher lets a channel that is structurally blind to a fault vote on it

Bugs found by measuring, not by reading

BUG	EFFECT
Lookahead in every detector	Scale statistics were computed over the whole test series, so October was normalised using December. Pressure PR-AUC fell 0.529 → 0.232 once fixed — the earlier numbers were inflated, and the code could not have served a live request at all
Injector labelled untouched points	A spike displaces half the samples in its window; all of them were labelled faulted. Naming accuracy rose 0.587 → 0.706 on the fix alone
Degenerate Mahalanobis axis (x3)	<code>flat</code> is zero for ~97 % of a healthy window, so its variance is ~0 and a distance weighted by 1/variance gave it 99.9 % of the score. The learned layer was a flatness detector in disguise

BUG	EFFECT
Fog is indistinguishable from a stuck probe	RH pinned at 100 % for hours is flat and at the rail either way. No shape descriptor separates them; the resolver is physical – fog wets the whole valley. 175 false saturation calls removed
20.5× neighbour gain	Came from a simulation whose neighbours shared a seasonal signal by construction. Measured on real data it is 1.4–2.3×

Current accuracy, stated plainly

VARIABLE	PR-AUC	EVENT RECALL	FALSE ALARMS / STATION-DAY (QUIET)
pressure	0.184	0.638	0.019
temperature	0.179	0.651	0.110
humidity	0.071	0.517	0.073

These are weak and they are not dressed up. What makes them defensible is that they are measured causally, on held-out stations and held-out months, at a fixed alert budget, with every regression traceable to a commit. False alarms in quiet weather and in active weather are within ~20 % of each other – which is the evidence that the system is separating faults from weather rather than hiding the problem.

Next, in order

- 1. Replace the basemap.** done OpenStreetMap's depiction of the boundaries of Jammu & Kashmir and Aksai Chin is not the official Government of India depiction. The map now draws Esri World Imagery under the **official NCMRWF boundary** – served as WMS tiles through the API's proxy, or as the vendored geometry where there is no proxy. Same source either way; the depiction of a national border is not something this application improvises.
- 2. Verify station coordinates.** Latitudes and longitudes are approximate. Longitude drives every diurnal baseline.
- 3. Improve per-class precision in the signature stage.** It is the blocker for per-fault-type budgets and it is where the explainability marks live.
- 4. Get real IMD data.** part 346 real IMD stations now come from WMO's WDQMS, with their observation- minus-background departures, and the simulated network sits at 344 of those real locations. The *readings* are still generated, and that is now a decision rather than a gap: the finals will run on simulated data too. The original note stands for everything below it – Everything rests on reanalysis, which is spatially smooth by

construction. Station density and usable correlation radius remain the project's largest unanswered risk.

5. **Edge tier.** `done` `firmware/skyguard_node` is an ESP32 sketch that runs end to end in Wokwi: level-0 screen on the node, housekeeping channels, TLS uplink to the deployed Space, and the panel returning a verdict on what it sent. It has never run on physical hardware.

Rebuilt since — what changed and what it cost

Everything above describes the system as first built. The work below replaced parts of it.

Where a number moved because a mistake was found rather than because the system improved, it says which.

WHAT CHANGED	WHY	MEASURED EFFECT
σ is estimated from a trailing, lagged window, not fitted once on an early one	The old estimate was in-sample — fitted to the window it was then scored on — and then frozen, so a month of changing weather was judged against a fortnight of one regime. On a network with <i>no faults injected at all</i> , $ z $ p95 was 2.78 in the baseline window and 4.47 later: “4 σ ” was really about 2.5 σ by the end.	Inflation 1.61 $\times$ → 1.23$\times$. False alarms at 4 σ 7.6% → 4.1%.
Bands calibrated against a fault-free control run, not asserted	4 and 6 were chosen against the old σ . Re-swept end to end — inject, grade blind, debounce — asking which <i>stations</i> raise an alert at all.	6 and 8 chosen: 57% recall at 38% precision , against 86% at 15% before. 72% recall once dropouts are excluded — a station that stops reporting has no residual to difference, so it is correctly NO DATA rather than a fault.
Alerts are episodes with persistence and hysteresis — raise after 3 consecutive flagged hours, clear after 6 clean ones	The grade is recomputed independently every step with no memory, so a flag was a statement about one instant. Nothing was ever “resolved”: the flag simply was not there next hour. 2,484 flagged runs, 1,318 of them exactly one hour long .	2,484 episodes → 73 . Stations alerting 275 → 42. Zero one-hour blips. Clearing is deliberately harder than raising, or a drifting sensor closes its own alert and reopens it an hour later.
Rails come from WMO-No. 8, quoted with citation	The plausible-range limits were ours. Vol. I Annex 1.A states them: −80...60 °C, 0...100 %, 500...1080 hPa, with required uncertainties of 0.1 K, 1 % and 0.1 hPa.	In <code>physics/wmo_limits.py</code> . RH stays at 105 % as a <i>named departure</i> , not a silent one. Step and persistence limits remain ours and are labelled as such.
The two frontends became one:	Two frontends that must be kept in step is a guarantee that one of them is wrong. It	One map, one clock, one station index. Charts draw as the clock runs rather than

WHAT CHANGED	WHY	MEASURED EFFECT
React + Vite, static console deleted	was the older one: its channel charts had collapsed to two pixels wide and nobody noticed, because nobody was looking at it.	pre-printing the month.
Deployed — API and dashboard on one Hugging Face Space	A free account cannot create a Docker or <code>cpu-basic</code> Space; the API answers 402. ZeroGPU is the only free way to get a Python process, so it runs there with a no-op <code>@spaces.GPU</code> function that is never called.	One origin, no CORS. There is no GPU in this design and none is ever requested.

What none of this fixed. 192 stations still raise an alert where 28 have a fault. Debouncing cleans up the presentation of a grader that over-flags; it does not make the grader right. And the largest unclaimed block on the rubric is still **real-time capability**: `POST /api/ingest` accepts a reading and screens it against the WMO rails, but nothing pushes the verdict anywhere, so the map replays a record instead of watching a stream. Closing that is the same code path the ESP32 will use.

Prior art, and what IMD has already asked for

Correction 4 — the central novelty claim was wrong. This page previously led with “a signature classifier that names WHICH fault” as the innovation, and asserted that nothing in standard QC does this. That is false. Naming sensor fault types is a fifteen-year-old research programme with its own benchmark datasets — Sharma, Golubchik & Govindan (ACM TOSN 6(3), 2010) define a fault taxonomy and detect by type on real environmental deployments; CAFD (Sensors 21(2):617, 2021) performs context-aware fault *diagnosis* on a wireless sensor network of temperature and humidity. The vocabulary we use — drift, bias, spike, erratic, stuck, data-loss — is the field-standard set. Cite this prior art first, before a judge does.

Correction 5 — and the gap framing was wrong too. A later draft of this section claimed we had found a gap IMD had not noticed. Reading IMD's own **Vision 2047** (Ministry of Earth Sciences, January 2025, 284 pp.) shows they have noticed, specified it, and budgeted for it. That is not bad news. It replaces a claim we could not defend with one we can.

What IMD has already written down

Under *Calibration of Instruments & Sensors*, Vision 2047 lists as its own short-, medium- and long-term objectives:

- “Integration of **automated calibration systems to ensure the accuracy of measurements over time**”
- “**Self-monitoring and diagnostic capabilities** to identify and address malfunction”
- “**Automated calibration planner**”
- “Calibration details as part of the **metadata of the sensor**”

Under *Surface Observational Network*:

- “**Smart Weather Stations: An intelligent, autonomous AWS with advanced sensors integrated with AI algorithms for real-time Quality Control (150)**”
- “Implementation of a **primary level data screening** and feedback to the operator about the quality of the data”

Under *Data-driven Modeling*:

- “It is also contemplated to use **AI in remote calibration and data quality assessment for sensors & systems and apply correction factors** before ingesting the data in models”

And listed among the gaps: “**Frequent validation and calibration of technical tools** – There must be frequent validation and calibration of sensors, AWS, ARGS and other technical tools.”

So the pitch is not “we found a gap”. It is: **IMD has specified what they want; nobody has specified how, or what it costs; we built it and measured it.** That is a smaller claim and a far harder one to knock down. It also positions the work as alignment with their roadmap rather than a critique of their system – which is the correct posture in front of the people who run it.

VISION 2047 OBJECTIVE	WHAT WE BUILT	MEASURED
“automated calibration systems ... accuracy of measurements over time”	Frozen-reference neighbour-difference detector at a conformal alert budget	0.01 K/day at recall 1.00, one alert per station-week – <i>on synthetic data; see the caveat below</i>
“self-monitoring and diagnostic capabilities”	Sticky per-probe health state with hysteresis, worst-channel station roll-up, kept separate from per-timestamp flags	Built and running in the console

VISION 2047 OBJECTIVE	WHAT WE BUILT	MEASURED
“automated calibration planner”	Not yet built. Theil–Sen slope extrapolated to tolerance, ranked by days-to-out-of-spec	The slope is already computed; only the projection is missing
“Smart Weather Stations ... AI algorithms for real-time Quality Control (150)”	Not yet built. Level-0 tier on ESP32-class hardware	Fitted model is ~6.5 kB and ~80 float operations per sample; streaming approximations validated against the batch versions
“primary level data screening”	The physics screen, written dependency-free so it transliterates to C++	Runs; not yet on a device

The one mechanism nobody has written down

A baseline refitted on data containing a slow degradation absorbs about 93 % of it and goes blind.

This is ours, it is measured, and it is the reason the objectives above are harder than they look. IMD's documented spatial check – the Spatial Regression Test of You, Hubbard & Goddard (2008) – refits its regression on a **25-hour window centred on the datum being tested**. A degradation that develops over months is inside that window by construction, so the reference bends to follow it. The check is not badly built; it is built for a different question.

We measured this three ways and were wrong twice before getting it right. It is not robustness – Huber absorbed more of the ramp than ordinary least squares (0.8 % versus 3.5 % surviving). It is not the neighbour difference – that recovered 2.4 %, worse than the raw residual. It is the freezing, and only the freezing: a baseline fitted on a clean reference period and never refitted on scored data leaves the degradation fully visible.

The adaptive anomaly-detection literature is built on the premise that adapting is good. When the thing being detected is degradation of the sensor itself, adaptation is self-masking. That is a rule with an ablation behind it.

And the same structure appears in NCMRWF's background check. Observations are tested against the previous cycle's model forecast – but that forecast assimilated the same station six hours earlier, and the one before that, and so on. The reference is not independent of the station's own history. A degradation of 0.01 K/day moves the observation and the background together, so the departure stays small and the probability of gross error never rises. Their specified SYNOP temperature error is 1.4 K; a drift takes almost five months to reach it, and by then the background has followed. This is not a flaw in their system – a background check exists to catch gross errors in a single cycle, and it does that well, rejecting under 1 % of temperature reports. It is simply the wrong instrument for a fault that develops slower than the assimilation window, and nothing in their documented pipeline is the right one.

What IMD's published system already does

One substantive public description exists: **Ranalkar, Anjan, Mishra, Mali & Krishnaiah, MAUSAM 66(1), January 2015, 93–106**, by IMD Pune, describing what was operational at the Central Data Receiving Earth Station. Read it before presenting anything. It runs range checks against sensor and monthly climatological limits; step checks at 1 and 3 hours with per-station monthly thresholds; internal consistency including $T < T_d - \gamma$; and the Spatial Regression Test described above. It defines a flag vocabulary, and it monitors the

network for non-functional and intermittently transmitting stations, battery voltage and GPS lock, emailed to the RIMCs.

That paper is from 2015 and we do not know what runs in 2026. No successor description of the core AWS QC algorithm was found. Vision 2047 sets short-term objectives for AWS-level quality intelligence (below), and sister organisations run their own quality control, and it is substantial. Read directly: **Desamsetti, Sharma, Rani & Das Gupta, “Monitoring and quality control of surface meteorological observations at NCMRWF”, NMRF/TR/02/2022.** Their Observation Processing System applies a **Bayesian gross-error framework**: an initial probability of gross error is updated by a consistency check (PGE1), a **background check against the previous cycle's model forecast** (PGE2), and a **buddy check against nearby observations of the same type** (PGE3), after which anything at $PGE3 \geq 0.5$ is not assimilated. For SYNOP the specified errors are 70 Pa, 1.4 m/s, 1.4 K and 7 % RH. Measured rejection over June–July 2021: under 1 % for temperature, wind and pressure in the background check and under 1.5 % in the buddy check, with humidity around 4 %.

Two claims relayed to this project from a secondary summary do not survive contact with the primary documents and have been removed: NCObsProM contains no “dynamic blacklisting” and no Pass/Valid/Conditional/Reject flag scheme — the words “blacklist” and “flag” do not occur in it at all. Its actual rule for Indian surface observations is a *volume monitor*: “if the total number of Indian surface observations received during each assimilation cycle is less than 20 % of the mean of the previous 45 days count, then action will be taken by informing the concerned agencies”. That counts arrivals; it does not examine values. And every occurrence of “drift” in the surface QC report refers to *drifting buoys*, not calibration drift.

Mission Mausam is largely orthogonal to this work, and an earlier draft of this page said otherwise. Its published deliverables (MoES, ₹2,000 crore, Phase-I 2024–26) are new instruments and cloud physics — 50 DWRs, 60 RS/RW, 100 disdrometers, wind profilers, radiometers, testbeds, 10 Marine AWS — plus a substantial weather modification programme. There is no AWS network expansion in the deliverables and exactly one quality-control line: “*Evaluating the existing observing network and preparing quality-controlled datasets for model verification.*” Its AI/ML objectives are about forecasts and earth system models, not observation QC. A secondary research summary attributes a four-stage QC pipeline with ML fault-versus-cloudburst classification to Mission Mausam; that is **not in the primary document** and should not be repeated. Treat every statement about what IMD does not do as “not present in the last public description”, and in the room **ask rather than assert.**

The honest form of our headline number. We detect degradation at 0.01 K/day with recall 1.00 at one alert per station per week — *on ERA5 reanalysis, degraded to AWS realism using noise parameters we chose ourselves*. The detectable slope scales directly with the neighbour-difference sigma, and we synthesised that sigma. On real station pairs it will be larger and we do not know by how much. Quote the number with its conditions attached every time, and present the latency-versus-slope curve rather than the single figure — a curve carries its own conditions, a number does not.

Stop claiming these

Each is operational in IMD's published system. Presenting any as an innovation is the fastest way to lose the room.

- Spatial or buddy checks between neighbouring stations — they run the SRT
- Range, gross-error or climatological limit checks
- Step or rate-of-change checks
- Internal consistency between temperature and humidity
- QC flags exposed to users — a vocabulary was defined in 2015
- Automated network and station health monitoring
- Interpolation of erroneous values
- And never that IMD has no quality control

Speak their vocabulary, not ours

Vision 2047 never uses the word “drift” in the sensor sense — not once in 284 pages. Their terms are *accuracy of measurements over time, correction factors, self-monitoring and diagnostic capabilities, automated calibration planner*. Use those. Elsewhere: **Level-0** and **Level-1 QC**, **SRT**, **ES** (Central Data Receiving Earth Station, Pune), **RIMC**, **NDC**, **SLP** for station level pressure and **MSLP** for the reduced value — IMD derives MSLP at the Earth Station, which answers a question this page previously listed as needing human verification. Their paper also notes that **10 m of elevation error produces about 1 hPa of error in MSLP**.

Where these sit in Vision 2047, and what the numbers actually say

The timeframe matters and it is favourable. Every objective this project maps onto is in **§1.1.3, Short-term Vision — next two years**, not a distant 2047 aspiration:

- **“Smart Weather Stations:** An intelligent, autonomous AWS with advanced sensors integrated with AI algorithms for real-time Quality Control (150)”

- “Integration of **automated calibration systems to ensure the accuracy of measurements over time**”
- “**Self-monitoring and diagnostic capabilities** to identify and address malfunction”
- “Implementation of a **primary level data screening** and feedback to the operator about the quality of the data”
- “Development of **Low-power weather stations** for deployment in remote, inhospitable or off-grid areas (300)” – a second, independent justification for the edge tier
- “Application of **AI/ML to pre-process the archived data and fill the gaps** in climate observation” – the problem statement's imputation objective, in IMD's own plan

The “*automated calibration planner*” is long-term; “*AI/ML-based primary-level data screening*” is medium-term.

FIGURE	WHAT VISION 2047 SAYS
Current network	Table 1: 1008 AWS , 200 Agro-AWS, 1382 ARG, 547 surface manual observatories, 6095 district rainfall stations
Reception	“Minimum data reception frequency is once every 15 mins for AWS and ARG with latency 10 mins ”. The GPRS server established 2018 supports 15-minute <i>or even 1-minute</i> intervals. State our per-record latency against that budget, not in the abstract
Scale, short-term	“district (800) & block level (1,50,000) ”
Scale, medium-term	“Automated Observational network at Panchayat level (2,50,000) ”

Their numbers are not internally consistent, so quote them carefully. §1.1.2 Gap Areas asks for “AWS at block level (6,612)” and “ARG at panchayat level (2,50,000)”, while §1.1.3 asks for an automated network at “block level (1,50,000)” and §1.1.4 for “Automated Observational network at Panchayat level (2,50,000)”. Whether the panchayat figure means rain gauges or full stations is ambiguous in the document itself. Quote the phrase verbatim with its section number rather than paraphrasing a count.

Open questions we have not resolved

- **Does the 2015 system include a persistence / flat-line check?** A direct read of the paper found four checks and no persistence test; a secondary research summary states persistence checks were present. The two conflict, and it matters because a flat-line

detector is one of the things we would otherwise claim. Resolve from the primary paper before saying anything.

- **Which flag vocabulary is IMD's?** A direct read gives 0 QC Not Done · 1 Good · 2 Doubtful · 3 Erroneous · 4 Changed/Interpolated · 5 Missing. A secondary summary gives an incompatible set. Using the wrong one in front of IMD would be a poor way to show we did the reading.
- **Data from the Data Supply Portal is post-processed.** Erroneous values appear to be filtered or replaced with missing markers and flag columns stripped from public CSV exports. If so, DSP data cannot be used to demonstrate anomaly detection – the anomalies have already been removed. Verify before paying for a dataset.
- **Inter-station spacing and usable correlation radius for the real network.** Still unanswered, still the project's largest risk. The SRT's 2°x2° search box is the only IMD-sourced hint.

References

Everything cited in this document, sorted by how safely it can go on a slide in front of MoES / IMD. Verification status is marked because it matters: **read** means the full text was opened and read during this work; **cited** means it is referenced but was not opened.

Primary — India Meteorological Department and MoES

SOURCE	LINK	
WMO-No. 8, Guide to Instruments and Methods of Observation (2024 ed.) — Vol. I Annex 1.A gives the operational range, resolution and required uncertainty this project rails against	library.wmo.int/records/item/68695	used
IMD Vision 2047 (January 2025, 284 pp.) — the roadmap this project aligns to	internal.imd.gov.in/press_release/20250114_pr_3552.pdf	read
Ranalkar, Anjan, Mishra, Mali & Krishnaiah, “Development of operational near real-time network monitoring and quality control system for implementation at AWS data receiving earth station”, MAUSAM 66(1), Jan 2015, 93–106 — IMD's own operational QC	mausamjournal.imd.gov.in/.../article/view/370	read
Vision 2047 for Cyclone Warning Services and Action Plan (Feb 2024, IMD / MoES / WMO)	IMD, February 2024	read
IMD Annual Report 2022	mausam.imd.gov.in/imd_latest/contents/ar2022.pdf	read

SOURCE	LINK	
Notes on AWS and ARG — IMD Pune training material (calibration practice)	impdune.gov.in/training/icitc/	read
Network of Automatic Weather Stations: TDMA type — MAUSAM	.../article/view/1048	cited
Network of Automatic Weather Stations: PRBS type — MAUSAM	.../article/download/412/327/1310	cited
Digital Automatic Recording System (DARS) — MAUSAM	.../article/view/411	cited
Climate Services in India: An Overview — IMD Pune (NDC / DSP)	impdune.gov.in/NFCS/	cited
IMD Data Supply Portal — user categories and tariffs	dsp.imdpune.gov.in/home_categories.php	cited
Procedure for Data Procurement — IMD	mausam.imd.gov.in/slimla/docs/data-procedure.pdf	cited
METNET newsletter (AWS deployment plans)	metnet.imd.gov.in/docs/imdnews/	cited
PIB — IMD 150th Foundation Day, Mission Mausam	pib.gov.in/PRID=2092861	cited
PIB — 50 AWS each in Delhi, Mumbai, Chennai, Pune (2026)	pib.gov.in/PRID=2214967	cited

Prior art — cite these before a judge does

WHAT IT ESTABLISHES	CITATION
Naming sensor fault types is not new	Sharma, Golubchik & Govindan, “Sensor Faults: Detection Methods and Prevalence in Real-World Datasets”, ACM TOSN 6(3), 2010. qed.usc.edu/papers/SharmaGG10.pdf
Fault <i>diagnosis</i> on a temperature + humidity sensor network	Saeed et al., “CAFD: Context-Aware Fault Diagnostic Scheme”, Sensors 21(2):617, 2021. doi.org/10.3390/s21020617
Relative homogenization — the offline ancestor of our drift channel	Menne & Williams, “Homogenization of Temperature Series via Pairwise Comparisons”, J. Climate 22(7), 2009. journals.ametsoc.org/

WHAT IT ESTABLISHES	CITATION
The Spatial Regression Test – the method IMD actually runs	You, Hubbard & Goddard, 2008 (as cited by Ranalkar et al. 2015)
A fielded operational spatial QC system	TITAN, Adv. Sci. Res. 17:153, 2020. asr.copernicus.org/articles/17/153/2020/
Continental-scale automated QA	Durre, Menne et al., NOAA/NCEI, 2010. ncei.noaa.gov/.../durre-menne-et al2010.pdf
Conformal prediction, for the alert-budget layer	Angelopoulos & Bates, “A Gentle Introduction to Conformal Prediction”. arxiv.org/abs/2107.07511

Indian operational practice beyond IMD

- **NCMRWF NMRF/RR/01/2021** – Saha, Das Gupta & Mitra, *Monitoring the Quality of AWS/ARG Rainfall Observations over India during Monsoon Season*. researchgate.net/publication/361760829
- **NCObsProM** – NCMRWF Observation Reception, Processing and Monitoring System. researchgate.net/publication/387834527
- **Telangana** – Real-time Rainfall Quality Monitoring for a Highly-Dense AWS Network. researchgate.net/publication/374761338
- **APSDPS (Andhra Pradesh)** – Ahmed Shoaib & Rasool, “Correcting Real Time AWS Data Through Quality Checks and Analysis”, *Vayu Mandal* 41, 2015. imetsociety.org/.../2015_9.pdf read

Context for the motivation slide

- **Mungeshpur, 52.9 °C, May 2024** – IMD confirmed a sensor reading about 3 °C high; caught by human comparison against neighbours after national publication. tribuneindia.com/...
- **IMD restricted public AWS/ARG portal access, May 2025** – affects any demo assuming a live public feed. downtoearth.org.in/...

Do not put these on a slide. Several secondary compilations of Vision 2047 circulate on UPSC coaching sites – understandupsc.com, gktoday.in, theiashub.com, pw.live. They are not sources, their numbers are second-hand, and citing them to IMD undercuts everything else on the page. The Vision 2047 PDF linked above is the primary document; use it. For the same reason, prefer DOI and journal links over ResearchGate mirrors wherever both exist.

What this is built on — and what is real

Every review of this project raised the same objection: there is no labelled corpus of real AWS sensor faults, so any recall figure is measured against faults we injected ourselves, which is circular. That is true for India. It is not true everywhere, and three real sources now sit underneath the work.

SOURCE	WHAT IT GIVES	STATUS
DOE ARM surface meteorology	Temperature, pressure and humidity together at 1-minute, plus <code>logger_volt</code> and <code>logger_temp</code> each with their own QC flag — and Data Quality Reports , human-written fault intervals with a stated cause and exact start and end times, graded Incorrect or Suspect	Downloaded. 542 daily files, 139 MB, stations at Eastern North Atlantic, North Slope Alaska and Southern Great Plains. The DQR labels need no account ; only the measurements do
WMO WDQMS	Per-station observation-minus-background departures for <i>exactly</i> our three variables, from four independent NWP centres, including Indian stations	Public REST endpoint, no authentication. 349 Indian stations harvested
NOAA USCRN raw transmitted stream	The three independent thermometers each station really carries, the fan-speed pulse rate per aspirated shield , battery channels, and minutes the datalogger door was open — none of which survive into the published product	Being mirrored nightly, because NCEI keeps only ~16 days and the rest is discarded continuously

What the real data changed

An earlier version of this analysis measured which thermodynamic quantities the atmosphere holds still — the measurement that decides which coordinate the whole system works in. It ran on reanalysis degraded by *our own* noise assumptions, so its magnitudes were partly a measurement of our own dictionary. Rerun on real 1-minute observations, filtered to quality-flag-clean rows outside any human-written fault interval:

QUANTITY	SYNTHETIC RATIO	REAL RATIO
q specific humidity	0.91	0.45
θ_e equivalent potential temp	1.07	0.45
Td dew point	0.87	0.60
T temperature	2.76	0.62
RH relative humidity	3.26	1.39

Ratio is the diurnal range within a day against the spread of daily means between days. A small ratio means the sun barely moves it, so a fault that does has nowhere to hide.

The synthetic figure overstated the advantage by about 60 % – it claimed working in dew point was 3.7× quieter than relative humidity; on real air it is 2.3×. The direction held, the magnitude did not, and that number would otherwise have gone on a slide.

It also changed the answer: on real observations the most conserved coordinates are specific humidity and equivalent potential temperature, both at 0.45, ahead of dew point. The synthetic run had put dew point first. The coordinate to difference is q , not T_d .

What the real fault reports settle

48 human-adjudicated fault windows touching our variables or the logger housekeeping, each with a cause and exact times. Four things that had been assumption are now observation:

WE HAD ASSUMED	THE REAL REPORTS SAY
Fault durations, guessed by our injector	Median 96 h , quartiles 20–196 h, longest 614 days . Ours: median 40 h. Same order, but reality's tail is far heavier than we generate
Clock drift is a clever hypothesis	<i>“Incorrect data logger clock”</i> is a written report. It is a documented fault class, which is what the pressure-tide work rests on
Housekeeping <i>might</i> carry fault signal	<code>logger_volt</code> and <code>logger_temp</code> each appear in 11 reports
Correlated fleet failure is a theoretical blind spot	<i>“Temperature measurement error”</i> recurs as the same subject at five different stations . The failure mode we argued is invisible to every neighbour-based method, IMD's included, is observable in someone else's real network

The real cause vocabulary also contains things our injector has never heard of: **ice storms**, **lightning strikes**, **system power issues**, **connection errors**.

Three caveats that travel with all of the above. ARM sites are research-grade and better maintained than a national operational network, so these fault rates are a lower bound on IMD's, not an estimate of them. The stations are North Atlantic, Alaskan and Great Plains — climatically nothing like India, so the conservation ratios will differ there and must be re-measured. And observation-minus-background departures are computed against a model that has already assimilated the station being judged, so they go blinder the slower the fault; our neighbour difference has the opposite bias, which is why the two are complementary rather than competing.

Measured so far

The sections above are design. These are results from the built pipeline, on ERA5 degraded to AWS realism with injected labelled faults. **They supersede any target stated above.**

RESULT	MEASURED	WHERE
Earned event recall, temp	0.281 → 0.942 by duration	<code>evaluation/run_placebo.py</code>
Earned event recall, pressure	0.365 → 0.979 by duration	<code>evaluation/run_placebo.py</code>
RH, faults longer than 16 d	-0.632 – detects none of them	<code>evaluation/run_placebo.py</code>
Signature naming accuracy	0.611, 33 % honest <code>unknown</code>	<code>evaluation/run_full.py</code>
<code>shield_failure</code> naming	not achieved	<code>evaluation/run_full.py</code>
Online IRLS vs batch IRLS	residual correlation 0.995–0.998	<code>evaluation/run_edge_approx.py</code>
P ² sketch vs exact median/MAD	3 σ agreement 99.2–99.4 %, 20 floats vs 720	<code>evaluation/run_edge_approx.py</code>
Drift inside the fitting window	absorbed by every estimator (0.008–0.064 recovered)	<code>evaluation/run_edge_approx.py</code>

RESULT	MEASURED	WHERE
σ , daily-mean neighbour difference	0.876 K	evaluation/run_cadence.py
Drift latency at 0.02 K/day, 3σ	131 days	evaluation/run_cadence.py
Cadence dependence, 15 min $\rightarrow$ 6 h	σ 0.875 $\rightarrow$ 0.921 K; latency 131 $\rightarrow$ 138 d	evaluation/run_cadence.py

The drift latency is weeks, not real time, and that is the honest answer. The drift section predicted exactly this; it is now confirmed rather than assumed.

Robustness does not protect a baseline from a drift in its own fitting window. Fitted over a window containing a 0.02 K/day drift, the fraction still visible in the residual is 0.035 for OLS, **0.008 for batch Huber**, 0.064 for online Huber – the robust estimator is the worst of the three, because a slow drift never looks like an outlier. Only a frozen reference window preserves the signal, which is why the station tier is sent its coefficients and never refits.

Cadence-independence matters for deployment. A 24 $\times$ change in sampling rate moves σ by 5 %, so the drift result describes a physical timescale and transfers to the 15-minute cadence real AWS report at. But **every window in the code is expressed in samples**: at 15-minute data the 720-sample scale window is 7.5 days rather than 30, and the 24-sample Hampel window is 6 hours rather than a full diurnal cycle – it would no longer contain the cycle it exists to normalise against. Any redeployment must rescale windows by *time*.

shield_failure is detected but cannot yet be named, and three fixes were measured and rejected. The fault is switched off by sunset, so the detector fires in short daily bursts and the classifier sees a 4-hour fragment rather than a 400-hour event. Adding a **diurnal_gain** descriptor changed nothing — an A/B against the pre-change classifier came back *identical* at 0.595 in both arms, because the template never fired at all. Repairing **td_ratio** to be measured on diurnal gain rather than the window median moved naming to 0.611 but still never named a shield fault. Widening the run-merge gap to 30 h finally let it fire, and cost more than it gained: naming fell to 0.489 as genuine 1-hour spikes merged into the quiet hours around them and read as weather. One global merge gap cannot serve both a spike and a sun-driven fault; that needs per-hypothesis merging, which is not built. **The honest state is: detected, not named.**

Caveat on the 15-minute arm: ERA5 has no sub-hourly variability to resample, so that arm is explicitly synthetic. It answers "do the windows scale", not "how good are we on real 15-minute data".

Design sections remain a blueprint and their success criteria are targets. The "Measured so far" table and the placebo table are measured. The Magnus tautology and the specific-humidity worked example were verified numerically; every empirical claim about tide amplitudes, correlation lengths, standard QC document numbers and component prices should be verified independently before it drives a decision or appears in a submission. Quoted statement text and scoring weights are from sih.gov.in.